

Compare the requests, not their sizes: Resource side channels and sound token bounds for LLM workflows

Nambi Rajan M^{a,*}

^a*Vellore Institute of Technology, Vellore, Tamil Nadu, India*

Abstract

A program that orchestrates calls to a large language model (LLM) can leak a secret without the secret reaching a prompt: if the secret decides which requests are sent, whoever sees the network traffic or the bill can learn it. Resource-aware noninterference handles such channels by charging each operation a cost determined by input sizes. We show that this cannot work for LLM calls. Any analysis that sees prompt text only through a size measure is either unsound for providers whose answers depend on content, or rejects a secret-guarded branch whose two arms are identical. The theorem is mechanised in Coq and explains why two size-based rules of our own failed. We compare requests by content instead, resolving each secret branch by the outcomes its secret can realise. If every resolution sends the same requests, the workflow is noninterfering for every provider, tokenizer and observer of its transcript; if k request classes remain, it leaks at most $\log_2 k$ bits of min-capacity. Because tokenization is not subadditive, we compute worst-case token bounds in bytes and convert them through contracts measured on fourteen tokenizers. Both analyses are implemented in OrchLang, a small workflow language, and evaluated on synthetic suites and on thirty workflows ported from LangGraph, the Anthropic cookbook and AgentDojo under a protocol fixed before porting. No certified bound was exceeded in 6,000 runs of the ported workflows. None of them has a secret, so the relational analysis has so far been exercised only on synthetic workflows.

Keywords: Information flow, Resource side channels, Relational analysis, Quantitative information flow, Tokenization, Large language models, Coq

*Corresponding author.

Email address: AUTHOR-EMAIL (Nambi Rajan M)

1. Introduction

Fig. 1 shows a customer-support workflow written in OrchLang, the language this paper is about. It answers a ticket with a small model. When the customer’s account carries a private fraud-risk flag, `high_risk`, it also asks a larger model to check the reply before the reply is sent. The flag never reaches a prompt, the workflow’s output or the tool that sends the reply. An information-flow type system with a program-counter label therefore has nothing to object to: nothing observable at a sink depends on the flag.

The requests do depend on it. For a customer with the flag set, the workflow makes two requests, one of them to a more expensive model; for everyone else it makes one. The provider bills per model, so a tenant’s invoice shows which customers were checked. An observer of the encrypted traffic sees request and response sizes, from which token-length and packet-size attacks have recovered response text and prompt topics [1, 2], and a prompt cache shared between users turns the same requests into a timing channel [3]. Each of these observers sees a function of the workflow’s requests and responses. If a secret changes the requests, it can change what they see, even when no secret value crosses any boundary.

1.1. Why the classical treatment does not transfer

This is a resource side channel, and its classical treatment is mature. Ngo, Dehesa-Azuara, Fredrikson and Hoffmann define resource-aware noninterference and constant resource, and certify both by combining information-flow typing with automatic amortised resource analysis (AARA) [4]. Çiçek, Barthe, Gaboardi, Garg and Hoffmann’s RelCost bounds the difference in cost between two executions with relational refinement types [5]. Ngo et al. quantify over environments that agree on the sizes of secret data, and RelCost bounds cost differences in terms of sizes and of how much two inputs differ. Both charge each operation a cost that the program’s data determine.

The cost of an LLM call is not determined by sizes. Two requests of exactly the same length in tokens can receive answers of different lengths, because the model answers what it is asked. We ran a small instruction-tuned model on fifteen pairs of chat requests padded to identical token counts, and under a shared sampling seed all fifteen pairs produced outputs of different lengths (Section 9.5). Tokenizers disagree with sizes as well: under the

```

1 workflow Triage budget 1000000 {
2   input ticket: text untrusted max_bytes 4096;
3   secret high_risk: boolean;
4   model small = mock("openai/gpt-4o-mini") max_tokens 300
5     tokenizer o200k_base overhead 9;
6   model large = mock("openai/gpt-4o") max_tokens 900
7     tokenizer o200k_base overhead 9;
8   prompt answer(t: text) -> text = "Answer this support ticket: {t}";
9   prompt check(r: text) -> text = "Check this reply for mistakes: {r}";
10  tool send_reply(r: text);
11
12  let reply: text = call answer(ticket) using small;
13  if high_risk {
14    let second: text = call check(reply) using large;
15  }
16  endorse(reply) as vetted: text
17    because "a support agent approves every reply";
18  emit send_reply(vetted);
19  output reply;
20 }

```

trriage.orch:13:3: error [E236] the guard 'high_risk' depends on a secret and the two arms
 send different requests, so the trace observer can tell which arm ran; one arm makes 1
 top-level request(s), the other 0. then-arm sends [openai/gpt-4o("Check this reply for
 mistakes: " + reply)] and else-arm sends [nothing]

Figure 1: The running example and the compiler’s verdict. The secret `high_risk` never reaches a prompt, an output or a tool, yet it decides whether the second request is sent. With the header `workflow Triage budget 1000000 leaks 1` the same program is accepted with a warning that it reveals at most one bit (Section 5.3); without the `endorse` on lines 16–17 it is rejected earlier, because text derived from the untrusted ticket would drive the tool (Section 3).

fourteen tokenizers we measured, 84–92% of random pairs of equal-length 35
 strings receive different token counts. One might hope to repair the classical 36
 analysis with a finer size measure. Section 4 shows that no size measure 37
 suffices: an analysis that sees prompt text only through a size measure is either 38
 unsound for some provider whose answers depend on content, or it rejects a 39
 secret-guarded branch whose two arms are identical (Theorem 1, mechanised 40
 in Coq). Instantiating Ngo et al.’s framework with a call charged by the sizes 41
 of its request yields such an analysis. RelCost can do better, because it can 42
 also state that two runs with identical inputs have the same cost; charging 43
 a call a zero difference exactly when its two requests are identical is sound, 44
 and it is, in essence, the comparison we develop in Section 5. Ngo et al.’s 45

quantitative bound fails for a different reason. It counts the possible values of a cost interval, and an LLM call may return anywhere from zero tokens to its cap, so the bound is at least $\log_2(\text{cap} + 1)$ bits for every workflow that calls a model, including those that provably leak nothing: 6.0 to 9.4 bits on the eleven leak-free workflows of our suite, where our bound is zero (Section 9.4).

We learned this twice. The first version of our compiler accepted a secret branch when its arms had equal certified cost bounds, and an external audit showed that equal maxima do not make equal bills. The repair compared symbolic sizes of requests; our own audit then showed that equal sizes do not make equal bills either. Section 4.1 records both counterexamples, and Theorem 1 explains why every repair of that kind fails.

1.2. Contributions

- *A size-blindness theorem* (Section 4). An analysis whose verdict depends on the text of a program only through a size measure is unsound for content-dependent providers or rejects identical arms, even if soundness is demanded only for deterministic providers and only for the total count of output tokens. We mechanised the theorem in Coq, and doing so exposed a gap in our own pen-and-paper proof.
- *Content-level request signatures* (Section 5). A branch guarded by secret content is resolved for every outcome vector the secret can realise, and the resulting request signatures are compared by content. If all resolutions agree, the workflow is noninterfering for every provider, tokenizer and observer of its transcript (Theorem 2, mechanised). If k classes remain, the channel from the secrets to any such observer has min-capacity at most $\log_2 k$, across any number of adaptively chosen invocations (Theorem 3). Observers that see less than the full transcript admit reordering of independent calls, except inside retry bodies (Theorem 4), and a rejection is justified up to dead code (Theorem 5).
- *Token bounds that hold for real tokenizers* (Section 6). Token counts are not subadditive, a response can re-encode to nine tokens per generated token, and six of fourteen tokenizers emit more tokens than bytes on some input. We therefore compute the guaranteed input bound in bytes and convert it once per request through per-tokenizer contracts generated from measurements. The bound is proved relative to those contracts (Theorem 6, mechanised).

- *An implementation and an evaluation able to fail* (Sections 8 and 9). The compiler `orchc` implements both analyses. Its runtime includes a provider whose answers depend on the request content, three ways of coupling the randomness of two runs, and re-billing with real tokenizers. The evaluation includes 30 real workflows ported from 52 candidates under a protocol that fixed the labels before any port was written, with a held-out split checked once against a frozen compiler.

We do not claim that combining information flow with resource analysis is new [4], nor relational cost analysis [5], nor leakage through the token counts, sizes and timing of LLM services [1–3, 6]. The flow part of OrchLang is conventional two-point confidentiality and integrity typing [7, 8]; for agents, CaMeL and FIDES enforce richer policies at run time [9, 10]. Accepting a secret branch because its arms perform the same observable operations goes back to Agat’s cross-copying transformation [11], and requiring the sequence of observable operations to be independent of the secret is the program-counter security model of Molnar et al. [12], with instructions replaced here by requests. What we add is the proof that, for LLM calls, “the same operation” has to mean the same request content, and an analysis built on that. Finally, the relational analysis never fires on the 30 real workflows, because none has a secret in its source’s threat model; its practical relevance rests on the threat model, not on deployed workflows (Section 9.6).

Section 2 fixes the model of providers, randomness and observers. Section 3 introduces OrchLang. Section 4 presents the two rules that failed and the theorem behind them, and Section 5 the analysis that replaces them. Section 6 turns to token bounds, Section 7 to the Coq development and Section 8 to the compiler. We evaluate in Section 9 and discuss related work in Section 10. Section 11 states the limitations and Section 12 concludes. The appendices contain the remaining proofs and the corpus of real workflows.

2. Setting

This section fixes what a provider may do, how randomness enters an execution, and who observes it. The guarantees of Section 5 are stated against these definitions; Section 2.4 lists what they leave out.

2.1. Workflows and providers

The analyses work on a core calculus that keeps the parts of OrchLang that matter for requests. A *model identity* m stands for everything that determines

an endpoint and its request parameters: provider, model name, output cap $cap(m)$ and client-side byte cap. A *request expression* $(m, t_0, a_1, t_1, \dots, a_n, t_n)$ is a prompt template with its arguments, where the t_j are literal text and each a_j is a literal or a variable. Commands are

$$c ::= \text{skip} \mid c_1; c_2 \mid x \leftarrow \text{call } r \mid \text{if } \pi(x) \text{ then } c_1 \text{ else } c_2 \mid \text{retry } n \ c \quad (n \geq 1),$$

where a guard π is a predicate on strings drawn from the forms the surface language allows: $\text{tokens}(x)$ compared with a constant, or the truth of a boolean. A store σ maps variables to strings. Variables are partitioned into public inputs, secrets and local bindings; block scoping is a well-formedness condition that the type system checks.

Definition 1 (Provider). A *provider* is a pair (Π, V) of functions. For a model identity m , a request text q and a random draw $\omega \in \Omega$, $\Pi(m, q, \omega)$ returns a response text and the number $o \leq cap(m)$ of output tokens the provider reports. For the transcript T of one retry attempt and a draw ω , $V(T, \omega)$ decides whether the attempt succeeded. We write $\mathcal{P}$ for the class of all providers.

Nothing else is assumed. Π and V may depend on every character of the request, which is the point: a real model answers what it is asked. A provider with state, such as a prompt cache or a rate limiter, is included as long as its state is a function of the requests it has received, since the results below all establish that two executions send the same requests.

2.2. Randomness and couplings

Randomness is supplied by a *tape* $\rho : K \rightarrow \Omega$ of independent draws indexed by keys. Each random event of an execution (a call, or the decision that ends a retry attempt) reads the tape at a key that a *keying scheme* computes from the execution so far. We use three schemes. The *global* scheme keys an event by its position in the execution; the *model* scheme keys a call by its model and the number of earlier calls to that model; the *request* scheme keys a call by its model, its request text and the number of earlier calls that sent the same text. A scheme is *fresh* if no two events of one execution read the same key. All three are fresh.

Lemma 1 (Coupling validity). *Under a fresh keying scheme, the transcript of an execution driven by a random tape has the same distribution as the transcript of an execution in which every random event draws independently from Ω .*

Table 1: Observers of an execution. Each row is a function of the row above it.

Observer	What it sees
trace	every event of the history, in order
provider	for each provider, its own events, in order
bill	for each model: number of calls, total input tokens, total output tokens
spend	the sum over models of price times tokens

Proof. The i -th event reads $\rho(k_i)$, where k_i is a function of the events before it and differs from $k_1, \dots, k_{i-1}$ by freshness. Conditioned on the first $i - 1$ draws, which determine k_i , the entry $\rho(k_i)$ has not been read, so it is independent of them and distributed as Ω . The transcript is the same function of the sequence of draws in both processes. $\square$

Running two executions on the same tape is therefore a proof device, a coupling in the sense of probabilistic relational verification [13, 14]. It never changes what a single execution does. The mock runtime of Section 8 implements all three schemes, so that the evaluation can check each verdict under each coupling.

2.3. Observers and noninterference

An execution produces a *history*, the sequence of events $\text{call}(m, q, \text{resp}, o)$ and $\text{attempt}(b)$. An observer is a function of the history. Table 1 lists the four we consider; each is a function of the one above it, so security against an observer implies security against every observer below it. The billed input of a call is $\tau_m(q) + \text{env}_m(q)$ for the model’s tokenizer τ_m and request envelope env_m , both arbitrary functions. A network observer who sees the sizes of encrypted requests and responses sees a function of the trace.

Two stores are *related*, $\sigma_1 \sim \sigma_2$, when they agree on every public input, including values that have been declassified.

Definition 2 (Noninterference). A program c is *O-noninterfering* for an observer O if for all related stores $\sigma_1 \sim \sigma_2$ the distributions of $O(h_1)$ and $O(h_2)$ are equal, where h_i is the history of c from σ_i with independent randomness. It is *pointwise O-noninterfering* under a keying scheme if $O(h_1(\rho)) = O(h_2(\rho))$ for every tape ρ .

175 This is the standard two-run formulation of noninterference [15, 16], with
 176 the observer made explicit. By Lemma 1, pointwise noninterference under any
 177 fresh scheme implies noninterference. When a workflow must reveal something,
 178 we measure how much with min-entropy leakage [17]: the logarithm of the
 179 factor by which an observation increases an adversary’s chance of guessing
 180 the secret in one try. The *min-capacity* of a channel is its largest min-entropy
 181 leakage over all prior distributions of the secret, and a workflow leaks at most
 182 b bits to O if the channel from the secrets to O ’s view has min-capacity at
 183 most b .

184 2.4. What the guarantee excludes

185 Timing is not part of the history, so it is covered only insofar as it is a
 186 function of the history, as with a cache keyed by requests. Providers whose
 187 randomness is correlated with the secret through a channel outside the pro-
 188 gram are excluded, as are observers who see more than the history, such as
 189 someone with access to a provider’s internal state. Declassification and en-
 190 dorsement are trusted: the compiler records the justification the programmer
 191 writes but checks nothing about it, and it does not enforce robustness in
 192 the sense of Myers, Sabelfeld and Zdancewic [18], so untrusted text could in
 193 principle influence what a declassification releases. Declassification is also all
 194 or nothing. A value declassified so that one provider may read it becomes
 195 public to every observer, which is coarser than the setting deserves when the
 196 provider is trusted with content but the network is not (Section 11).

197 3. OrchLang

198 OrchLang is a small, statically typed language for LLM workflows. A
 199 workflow declares its inputs, secrets, models, prompt templates and tools,
 200 then calls models, branches, retries, invokes tools and returns one output
 201 (Fig. 2). The compiler, `orchc`, makes no network requests; models are named
 202 by provider and model name and are only ever called by the mock runtime
 203 described in Section 8.

204 3.1. Design commitments

205 Three decisions make the analyses decidable, and Section 9.6 measures
 206 what they cost. First, the shape of a workflow is fixed before it runs: the only
 207 loop is `retry  $n$` with a literal n , and the only branch tests a declared value.
 208 Second, every length the compiler cannot see is declared in a stated unit:

```

w ::= workflow X budget n [leaks b] [observer O] { d* s* }
d ::= input x: T [untrusted] ℓ*; | secret x: T ℓ*;
    | model x = mock("provider/name") max_tokens n
      [tokenizer τ] [overhead n] [max_bytes n];
    | prompt p(x1: T1, ...) -> T = "template with {xi}";
    | tool f(x1: T1, ...);
ℓ ::= max_bytes n | max_tokens n tokenizer τ | max_tokens n
s ::= let y: T = call p(a1, ..., ak) using x;
    | emit f(a1, ..., ak); | output a;
    | if g { s* } [else { s* }] | retry n { s* }
    | (declassify | endorse)(x) as y: T because "justification";
g ::= x | tokens(x) ⋈ n      (⋈ ∈ {<, <=, >, >=, ==, !=})
a ::= x | "literal"

```

Figure 2: Abridged syntax of OrchLang. The full grammar, including **require** statements and monetary prices, is in the language specification that accompanies the artifact.

bytes (**max_bytes**), tokens of a named tokenizer (**max_tokens** *n* **tokenizer** τ), or, for an estimate only, tokens. Third, leaving the security lattice requires a written justification (**declassify**, **endorse**), which the compiler copies into its certificate.

Every declaration receives a unique binding identity, and all analyses refer to bindings, models and prompts by identity, never by name. A redeclaration inside a branch arm is therefore a different binding. This choice was forced by one of the counterexamples in Section 4.1. Inputs and secrets may be declared only at the top level of a workflow, since an input declared inside an arm would have no single meaning for the environment that supplies it.

3.2. Labels

Values carry labels from the product lattice $\{P \sqsubseteq S\} \times \{T \sqsubseteq U\}$ of confidentiality (public, secret) and integrity (trusted, untrusted) [7]; we write $C(\ell)$ and $I(\ell)$ for the two components and $\sqcup$ for the join. A literal and an ordinary input are (P, T), an input declared **untrusted** is (P, U), and a secret is (S, T). A model's response is labelled with the join of everything that reached its request, so text from a retrieved page stays untrusted through any number of calls. It can drive an effect only after someone endorses it and writes down why.

Each binding also carries a *data label*, which records what its content depends on, separately from the program counter *pc* under which it was computed. The distinction matters inside a secret-guarded arm. A program-

$$\begin{array}{c}
\frac{\Gamma(a_i) = (\ell_i, \delta_i) \quad C(\delta_i) = P \quad (1 \leq i \leq k) \quad [\text{E230}]}{\Gamma; pc \vdash \text{let } y = \text{call } p(a_1, \dots, a_k) \text{ using } m \quad \neg \Gamma[y \mapsto (pc \sqcup \bigsqcup_i \ell_i, (P, l(pc)) \sqcup \bigsqcup_i \delta_i)]} \text{CALL} \\
\\
\frac{\Gamma(x) = (\ell, \delta) \quad \Gamma; pc \sqcup \ell \vdash s_1 \quad \Gamma; pc \sqcup \ell \vdash s_2}{\Gamma; pc \vdash \text{if } x \{s_1\} \text{ else } \{s_2\} \neg \Gamma} \text{IF} \\
\\
\frac{\Gamma(a_i) = (\ell_i, \delta_i) \quad \ell_i \sqsubseteq (P, T) \quad [\text{E232, E233}] \quad pc \sqsubseteq (P, T) \quad [\text{E234, E235}]}{\Gamma; pc \vdash \text{emit } f(a_1, \dots, a_k) \neg \Gamma} \text{EMIT} \\
\\
\frac{\Gamma(a) = (\ell, \delta) \quad C(\ell \sqcup pc) = P \quad [\text{E231}]}{\Gamma; pc \vdash \text{output } a \neg \Gamma} \text{OUTPUT}
\end{array}$$

Figure 3: Selected label rules. Γ maps each binding to its label ℓ and data label δ ; literals are (P, T) . A premise that fails produces the diagnostic in brackets. In **CALL** the data label ignores the confidentiality of pc but keeps its integrity. In **IF**, a guard whose data label is secret also creates the relational obligation of Section 5.

231 counter discipline labels every value computed there as secret, so the second
 232 call of a chain inside the arm could not receive the first call's response. Yet
 233 the content of that response depends only on public data; only its existence
 234 depends on the secret, and existence is exactly what the relational analysis of
 235 Section 5 accounts for. OrchLang therefore checks prompt arguments against
 236 the data label, while outputs and effects, whose occurrence is visible outside
 237 the request trace, are still checked against the program counter. Fig. 3 gives
 238 the rules for calls, branches, effects and outputs.

239 The data label keeps integrity joined with the program counter, which
 240 is conservative: a value computed inside an arm guarded by untrusted data
 241 is untrusted even if it was computed only from trusted data. The real-
 242 workflow evaluation found two cases where this over-reports (Section 9.6).
 243 Reclassification copies a value into a new binding and relabels one component:
 244 **declassify** lowers confidentiality to that of the program counter and **endorse**
 245 lowers integrity to that of the program counter, so relabelling inside a secret
 246 arm still yields a secret-dependent value. An endorsed copy of a secret keeps
 247 a secret data label. Because the only bindings with secret data labels are
 248 secrets and their endorsed copies, and **CALL** keeps both out of requests, secret
 249 content never appears in a request.

250 **Example 1.** In Fig. 1, **reply** is (P, U) because the untrusted **ticket** reached

its request. Inside the arm, $pc = (S, T)$, so **second** gets label (S, U) but data label (P, U): another call could still receive it. Without the **endorse** on lines 16–17, EMIT fails with **E233**, because text derived from the ticket would choose what the tool sends. With it, **vetted** is (P, T) and the flow check passes; the branch on **high_risk** is then the relational analysis’s concern.

A workflow that passes every check receives a certificate: a JSON document bound to the source by its SHA-256 digest, which records the token bounds and their derivation, every label, every reclassification with its justification, and the leakage report. The certificate is a report, not a proof object; no independent checker exists yet (Section 11).

4. Why comparing sizes cannot work

A relational analysis for the channel of Fig. 1 has to decide when two arms of a secret branch are “the same” from the point of view of an observer. We tried two answers before the one in Section 5. Both compared something coarser than what determines the observation, and this section proves that every answer of that kind fails.

4.1. Two rules that failed

Equal bounds. The first rule accepted a secret branch when its arms had equal certified worst-case costs. An upper bound constrains a maximum, and two quantities with the same maximum need not be equal. The counterexample from the external audit is `if tokens(s) == 0 { call p(x) } else { call p(y) }`, with x and y both declared to hold at most 100 tokens. Both arms have the same bound, 111 tokens, and the bills differ whenever x and y differ in length; in the audit they differed in 225 of 425 paired runs. The workflow is still in the regression suite, where it is rejected and still leaks under the runtime.

Equal sizes. The repair abstracted each arm to a sequence of `Call( $m$ ,  $size$ )` nodes, with sizes symbolic in the lengths of variables, and accepted a branch whose arms gave equal sequences. It was unsound for three independent reasons, each with a counterexample that the rule accepts and the runtime shows leaking.

- *Tokenizers.* Sizes were estimated tokens, *bytes*/4. Real tokenizers bill strings of equal size differently: “aaaa” is one token under OpenAI’s

284 `r50k_base` encoding and `"bbbb"` is two. Re-billing the requests of the
 285 rule's four accepted counterexamples with `r50k_base`, `cl100k_base` and
 286 `o200k_base`, with the provider's answers held fixed, shows the bill moving
 287 with the secret for three of them.

- 288 • *Providers*. Even an exact token count does not help, because the provider
 289 answers the content of a request, not its size. The model of Section 9.5
 290 answered all fifteen pairs of equal-length requests at different lengths, 34.1
 291 tokens apart on average.
- 292 • *Names*. The rule compared models and prompts by their local names, so
 293 a redeclaration of either inside one arm was invisible to it.

294 On the relational suite of Section 9.3, the size rule accepts 12 workflows,
 295 four of which leak, and the bounds rule accepts 15, six of which leak.

296 4.2. The size-blindness theorem

297 The two rules have something in common: whether they accept a program
 298 depends on its text constants only through a measure of their size. We show
 299 that no rule of that form can be both sound and useful.

300 **Definition 3** (Size-indexed analysis). A *size measure* is a function $\mu : \Sigma^* \rightarrow \mathbb{N}$,
 301 such as the length in bytes, in characters, in estimated tokens, or in tokens
 302 under some tokenizer. A relational analysis A , a predicate on programs, is
 303 μ -indexed if $A(c) = A(c')$ whenever c' is c with one text constant t replaced by
 304 a string t' with $\mu(t') = \mu(t)$. The measure has *fresh twins* if for every string
 305 t with $\mu(t) > 0$ and every finite set W of strings, some t' with $\mu(t') = \mu(t)$
 306 contains a character that occurs in no member of W .

307 Every measure used in practice has fresh twins: strings of a given size can
 308 be built from characters the program never uses. Both withdrawn rules are
 309 μ -indexed, the first for the measure behind its cost bounds and the second
 310 for *bytes/4*.

311 **Theorem 1** (Size blindness). *Let μ have fresh twins, and let A be μ -indexed*
 312 *and sound in the following weak sense: for every program A accepts and every*
 313 *deterministic provider that respects the caps, any two related stores give the*
 314 *same total number of output tokens. Let c be a well-typed program with a*
 315 *secret-guarded branch, both of whose outcomes are feasible, whose then-arm*
 316 *unconditionally calls a model m with $\text{cap}(m) > 0$ on a request that contains*

a text constant t with $\mu(t) > 0$. Then A rejects c , whatever the else-arm is, including a copy of the then-arm.

Proof. Let W be the set of text constants of c , and choose a fresh twin t' of t that contains a character $\star$ occurring in no member of W . Let c' be c with t replaced by t' in that one call. Since A is μ -indexed, $A(c') = A(c)$. Consider the deterministic provider $\Pi^\star$ that answers the empty string to every request, reports $\text{cap}(m)$ output tokens when the request contains $\star$ and zero otherwise, and whose validator always accepts. Take every public input to be the empty string. Every request of c' is then a concatenation of members of $W \cup \{t'\}$ and empty strings, because secrets never reach a request (the CALL rule of Fig. 3) and every response is empty. So a request contains $\star$ exactly when it comes from the modified call. An execution whose secret selects the then-arm reports at least $\text{cap}(m) > 0$ output tokens; one whose secret selects the else-arm reports none. The two stores are related, their totals differ, and $\Pi^\star$ is deterministic and respects the caps, so A cannot soundly accept c' . Since $A(c) = A(c')$, it rejects c . $\square$

The theorem is mechanised as `size_blind` in our Coq development (Section 7), where “unconditionally” is rendered as “the call is in the top-level sequence of the then-arm”. It is the formal content of both withdrawn rules. Each compared something coarser than what determines the observation, a maximum in one case and a size in the other, and the proof builds a provider that exploits the difference. The theorem also says more than “these two rules were wrong”: no analysis that compares only sizes can be sound against content-dependent providers without rejecting a program as harmless as `if s { p(x) } else { p(x) }` with a non-empty template.

Mechanising the theorem exposed a gap in our first proof. The draft defined a fresh twin as a string that occurs in no member of W and in which no member of W occurs. That does not stop a *concatenation* of constants from containing the twin across a boundary: with $W = \{\mathbf{ab}\}$ and $t' = \mathbf{ba}$, the request `abab` contains t' . The Coq proof needs a character absent from every constant, and Definition 3 now requires one.

4.3. The classical instantiation

Ngo et al.’s constant resource and resource-aware noninterference relate executions whose environments agree on public data and on the *sizes* of secret data whose size is deemed public [4, Definitions 1 and 2], and their

type system attaches potential to data according to its size and charges each operation a constant cost. Instantiating it with an LLM call, charging the call a function of the sizes of its template and arguments and its output as a constant or as an unknown in $[0, cap]$, gives an analysis whose verdict depends on text constants only through their sizes.

Corollary 1. *The size-based instantiation of resource-aware noninterference is unsound for some content-dependent provider, or it rejects every secret-guarded branch whose then-arm calls a model with a non-empty prompt.*

The alternative, treating the cost of a call as data that depends on the request content, makes every call inside a secret branch a secret-dependent cost, and constant-resource typing then rejects the branch outright. The same happens under the standard program-counter rule if calls are treated as observable events.

Proposition 1. *If calls are typed as low-observable events with the program-counter rule of Volpano, Smith and Irvine [8], every secret-guarded branch that contains a call is rejected. If calls are treated as unobservable, the channel of Fig. 1 is not detected at all.*

RelCost is not size-indexed in the sense of Definition 3. Its relational refinements also record how much the inputs of two runs differ, for lists the number of positions at which they differ [5], so a call can be given a zero cost difference exactly when its two requests are identical. That instantiation is sound under the coupling of Section 2.2, and it amounts to comparing requests by content for a fixed pair of runs. What it does not provide, and Section 5 adds, is the resolution of a secret branch over the outcomes the secret can realise (RelCost relates two given runs), observers coarser than the trace, a leakage bound in bits, and an account of why comparing sizes cannot work. Table 2 summarises the options.

Ngo et al.’s quantitative bound fails differently. For uniformly sampled environments, it bounds the Shannon entropy of a program’s resource observations by $\log_2(u - l + 1)$, where u and l are upper and lower bounds on the cost [4, Section V-A, Lemmas 6 and 7]. An LLM call may return anywhere from zero tokens to its cap, so $u - l \geq cap$ for any workflow with a call, whether or not it has a secret, and the bound is at least $\log_2(cap + 1)$ bits for a workflow that provably leaks nothing. Counting observations is hopeless here, because the provider’s noise alone produces a vast number of possible observations; Section 5.3 counts request classes instead.

Table 2: Four ways to treat an LLM call inside a secret-guarded branch.

Calls are treated as	Consequence
unobservable	unsound: the channel of Fig. 1 is invisible (Proposition 1)
low-observable events	every secret branch containing a call is rejected (Proposition 1)
costs indexed by size	unsound for content-dependent providers, or identical arms are rejected (Theorem 1)
requests compared by content	sound for every provider, tokenizer and observer of the transcript (Theorem 2); rejections justified up to dead code (Theorem 5)

5. Comparing requests

The analysis that replaces both withdrawn rules compares what an observer of the transcript actually receives: the requests, by content. This section defines request signatures, resolves secret branches over the outcomes a secret can realise, and proves noninterference, a leakage bound, results for coarser observers and a completeness result.

5.1. Request signatures and resolution

Definition 4 (Request signature). The *signature* of a region of a workflow is a list of nodes

$$\begin{aligned}
 \text{node} &::= \text{Call}(m, \text{pieces}) \mid \text{Retry}(n, \text{sig}) \mid \text{Branch}(g, \text{sig}, \text{sig}) \\
 \text{piece} &::= \text{text}(t) \mid \text{var}(b) \mid \text{res}(p),
 \end{aligned}$$

where m is a model identity, $\text{text}(t)$ is static text as sent (template text and literal arguments, with adjacent text merged), $\text{var}(b)$ is a binding made outside the region, identified by its declaration and with public content, and $\text{res}(p)$ is the response of the call at position p inside the region.

A branch whose guard has public content contributes a **Branch** node with its guard term. A branch whose guard reads secret content is *resolved*. Because only secrets and their endorsed copies carry secret content, and the **CALL** rule keeps both out of requests, every secret guard is a predicate of the secrets alone. For a secret s with declared bound B , the guards $\text{tokens}(s) \bowtie k$

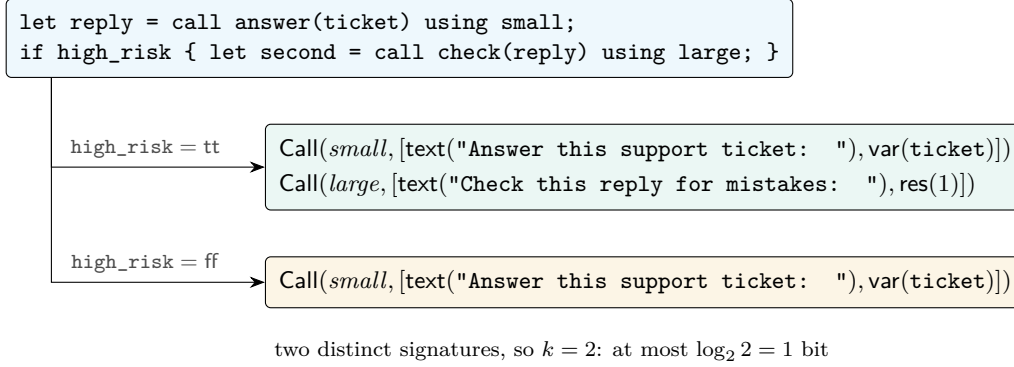


Figure 4: Resolution of the running example. The guard `high_risk` reads secret content, so the analysis inlines each arm the secret can select. The two resulting signatures differ in their second call, so $k = 2$: the workflow is rejected with E236, or accepted with a one-bit budget (`leaks 1`). The response of the first call appears as `res(1)`, not as text, because its content is the same in both executions whatever it is. (Diagram drawn in TikZ with the help of Claude, Anthropic.)

partition $[0, B]$ into intervals, and a boolean secret has two values. The analysis evaluates each guard at every value that could change an outcome, collects the outcome vectors that some value realises, and takes the product over independent secrets. The resulting set F is exactly the set of *feasible* outcome vectors. For $v \in F$, $sig_v(c)$ is the signature of c with every secret guard resolved by v , that is, with the selected arm inlined, and

$$k(c) = \left| \{ sig_v(c) : v \in F \} \right|$$

is the number of distinct signatures. The implementation computes $k(c)$ up to the normal forms of Section 5.4 for the observer the workflow declares.

Example 2. Fig. 4 resolves Fig. 1. The first call contributes

```
Call(small, [text("Answer this support ticket:  "), var(ticket)])
```

to both resolutions. With `high_risk = tt` the second call is inlined with the piece `res(1)` for `reply`; with `ff` nothing is. The two signatures differ, so $k = 2$. The compiler's message in Fig. 1 prints exactly this difference.

5.2. Noninterference

Lemma 2 (Equal signatures, equal histories). *Let $sig_{v_1}(c_1) = sig_{v_2}(c_2)$, where v_i is the outcome vector of store σ_i , and let σ_1 and σ_2 agree on every binding*

that occurs in a **var** piece. Then for every tape and every common history, running c_1 from σ_1 and c_2 from σ_2 under the global, model or request scheme extends the history by the same events, and binds equal values at corresponding positions.

Proof sketch. By induction on the signature. At a call, equal pieces denote equal text: **text** pieces directly, **var** pieces by hypothesis, and **res** pieces by the induction hypothesis. The same request therefore goes to the same model from the same history, the scheme computes the same key, the tape supplies the same draw, and Π returns the same response. At a retry, equal bodies produce equal attempt transcripts, and V reads the same transcript and draw, so both executions stop after the same attempt. A public branch reads equal values and takes the same arm. A resolved branch inlines the arm that the store actually takes, since v_i is σ_i 's own outcome vector. The full proof is in Appendix A. $\square$

Theorem 2 (Request-trace noninterference). *If $k(c) = 1$, then c is pointwise **trace**-noninterfering under the global, model and request schemes, and hence O -noninterfering for every provider in $\mathcal{P}$, every tokenizer, every envelope and every observer O that is a function of the history.*

Proof. Take related stores $\sigma_1 \sim \sigma_2$ with outcome vectors v_1, v_2 . Both vectors are feasible, so $\text{sig}_{v_1}(c) = \text{sig}_{v_2}(c)$ when $k(c) = 1$. For the whole workflow every **var** piece is a public input or a declassified value, on which the stores agree. Lemma 2 with the empty history gives equal histories on every tape, Lemma 1 turns this into equal distributions, and every observer is a function of the history. $\square$

No assumption about tokenizers is used. Equal histories contain equal request texts, and any tokenizer bills equal texts equally; that is the practical difference from comparing sizes, which is only as good as a tokenizer assumption and, by Theorem 1, not even that good. In the terms of probabilistic relational Hoare logic [13], Lemma 2 is a derivation in which every provider call is related by the identity coupling on its draw. The sampling rule that justifies such a step requires the two distributions to be equal up to a bijection, and with the identity bijection they must be identical. Requests with equal content satisfy this; requests with equal size do not, and read as such a derivation the size rule applies the rule to two different distributions. More broadly, comparing two resolutions is a specialised form of self-composition [19–21] in

456 which the product program never has to be built, because the signatures are
 457 compared directly.

458 5.3. Quantitative leakage

459 Rejecting every workflow with $k(c) > 1$ is too strict for workflows that
 460 must reveal a little, such as a service tier that selects a larger model for paying
 461 customers. A workflow may declare a budget **leaks** b , and it is accepted with
 462 warning W238 if $k(c) \leq 2^b$.

463 **Theorem 3** (Leakage bound). *For every provider in $\mathcal{P}$, every fresh keying
 464 scheme and every observer of the history, together with the workflow’s outputs
 465 and effects, the channel from the secrets to the observer has min-capacity at
 466 most $\log_2 k(c)$. The bound holds for any number of invocations with the same
 467 secrets and any public inputs, including public inputs chosen adaptively from
 468 earlier observations.*

469 *Proof sketch.* Partition the secret assignments by the class of their signature;
 470 there are at most $k(c)$ classes. By Lemma 2, two assignments in one class
 471 produce equal histories on every tape and for every public input, since
 472 signatures are symbolic in the public inputs. Outputs and effects are functions
 473 of public data and responses, because the type system rejects any output or
 474 effect under a secret program counter (Fig. 3). Each invocation’s observation
 475 is therefore a function of the class, the public inputs and the tape, and over
 476 adaptively chosen invocations the whole observation sequence is a function of
 477 the class, the tapes and the adversary’s coins. The channel is a cascade whose
 478 first stage is a deterministic map onto at most $k(c)$ classes. The min-entropy
 479 leakage of a cascade is at most that of its first stage [22], and for every prior
 480 the min-entropy leakage of a channel with at most k possible outputs is at
 481 most $\log_2 k$ [23, Theorem 1]; see also [24]. Appendix A gives the details. $\square$

482 The argument follows Köpf and Basin’s model of an adaptive attacker as
 483 a partition of the secrets into classes the attacker cannot tell apart [25], with
 484 signature classes in place of observations. Counting observations would be
 485 useless here, since the provider’s noise gives every workflow with a call a vast
 486 number of possible observations; that is why the interval bound of Section 4.3
 487 is vacuous. The coupling removes the noise from the count.

5.4. Coarser observers, and an unsound optimisation

A bill does not record order, and a provider sees only its own requests. The *provider normal form* of a signature stably sorts, by provider, every maximal run of calls none of which references another in the run, outside any retry body, and rewrites `res` references to follow the moved calls. The *bill normal form* sorts such runs by the full request.

Theorem 4 (Coarser observers). *If the provider (bill) normal forms of $\text{sig}_v(c)$ agree for all feasible v , then c is pointwise **provider-** (**bill-**) *noninterfering* under the request scheme, and hence **provider-** (**bill-**) *noninterfering*.*

Under the request scheme a call's draw is keyed by its request and occurrence, not by its position, so a stably sorted run sends the same multiset of requests and each copy of a request receives the same response. Calls inside a retry body are not reordered: the validator sees an attempt's transcript in order, and it may depend on the order. An intermediate version of our analysis did reorder there. It accepted a workflow for which, under the same seed, the runtime made one attempt for one value of the secret and three for the other. The workflow is now a regression test.

5.5. Completeness relative to dead code

Theorem 5 (Relative completeness). *Suppose $\text{sig}_{v_1}(c) \neq \text{sig}_{v_2}(c)$ for feasible v_1, v_2 , and every public guard on the path to the first difference can go either way for some public inputs and responses. Then some provider in $\mathcal{P}$ distinguishes the two secrets. For $k(c)$ classes, some provider and prior attain the bound of Theorem 3.*

Proof sketch. Choose public inputs that are distinct fresh strings, and a provider whose responses are fresh strings encoding the request and the draw, with a validator that always fails. Rendering signatures to histories is then injective, and the histories agree up to the first difference by Lemma 2. $\square$

A rejection is therefore justified, except when the difference sits under a public guard that can never go the relevant way. One consequence concerns two different public variables of equal declared length. A size-based rule accepts a branch that sends one in each arm; the content rule rejects it, and Theorem 5 says the rejection is required, because the variables can hold different text and some provider answers them differently.

521 6. Token bounds that hold for real tokenizers

522 A worst-case bound on the tokens a workflow is billed for has an output side
 523 and an input side. The output side is easy, because providers enforce a model's
 524 output cap themselves. The input side is where the obvious composition,
 525 $tokens(template) + \sum tokens(argument)$, fails.

526 The combinators of our bound are the textbook ones for a structured
 527 language [26]: a sum for sequencing, a maximum for a branch, and multi-
 528 plication for a bounded loop, as in cost-relation analyses of bytecode and of
 529 workflow models [27, 28]. What is new is the measure. In a workflow model
 530 such as Rpl the cost of a step is declared by the program [28]; in an LLM
 531 workflow the cost of a call is decided by a tokenizer the program does not
 532 control, applied to text the program only partly knows.

533 6.1. What fails

534 We measured fourteen tokenizers, each pinned to an exact revision: GPT-2,
 535 the `r50k_base`, `cl100k_base` and `o200k_base` encodings of tiktoken [29], and
 536 the tokenizers of Llama 2 and 3, Mistral, Phi-3, Gemma 2, Qwen2.5, DeepSeek-
 537 V3, OLMo 2, T5 and XLM-R loaded through Hugging Face `tokenizers` [30].
 538 The inputs were every Unicode scalar value, 388 texts of the Universal
 539 Declaration of Human Rights in 360 languages [31], twelve source files and
 540 eight JSON files. Four facts emerged.

- 541 1. *Tokenization is not subadditive.* $tokens(u \cdot v) - tokens(u) - tokens(v)$ reaches
 542 3 to 6, depending on the tokenizer. Under `cl100k_base`, " `Attribute`"
 543 and " `profiles`" are one token each, and " `Attributeprofiles`" is six.
- 544 2. *Responses inflate when re-encoded.* Decoding generated tokens to text and
 545 encoding the text again with the same tokenizer yields up to nine tokens
 546 per generated token (Qwen2.5), so a model's output cap does not bound
 547 the length of its response as input to the next call.
- 548 3. *Tokens can outnumber bytes.* The bound $tokens(s) \leq bytes(s)$ fails for six
 549 of the fourteen tokenizers. SentencePiece tokenizers add a dummy prefix
 550 that no byte pays for, Qwen2.5 normalises its input to NFC and turns the
 551 three-byte character U+0FAC into six tokens, and the normalisers of T5
 552 and XLM-R, which resemble NFKC, produce up to 194 more tokens than
 553 bytes on one string.

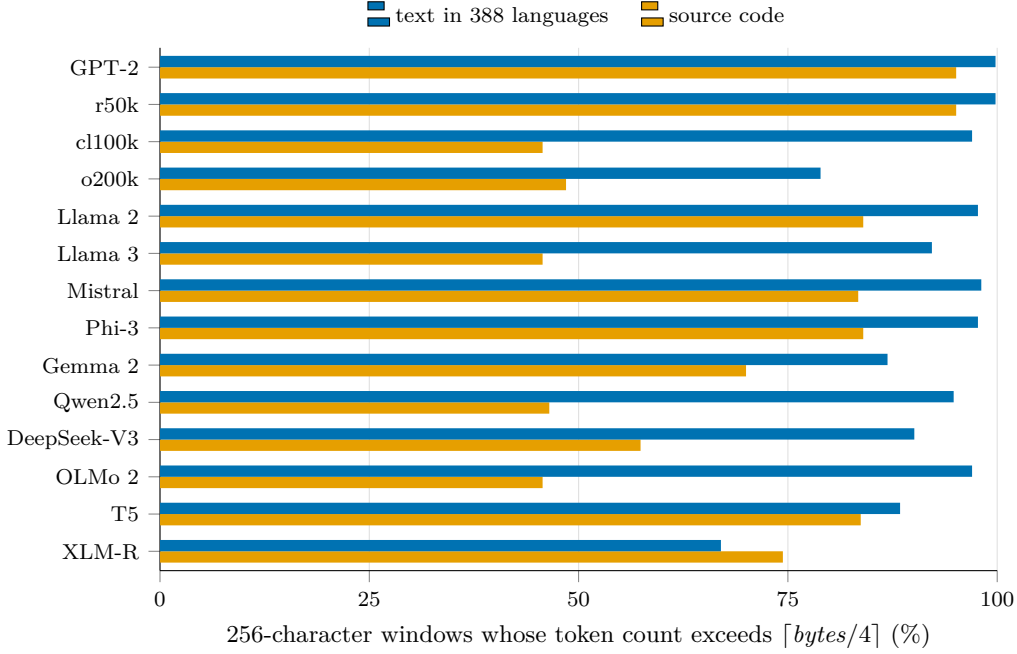


Figure 5: Share of 256-character windows whose token count exceeds $\lceil \text{bytes}/4 \rceil$, for each of the fourteen tokenizers. The estimate fails on most multilingual text for every tokenizer, and on almost all source code for the older byte-level encodings (GPT-2, r50k) and the SentencePiece tokenizers. The data are those of `bench/results/tokenizers.json`.

4. *The usual estimate is only an estimate.* A compiler can compute $\lceil \text{bytes}/4 \rceil$, and that estimate is exceeded on 67.0–99.8% of 256-character windows of the multilingual texts and on 45.7–95.1% of windows of source code, depending on the tokenizer (Fig. 5).

These are not new observations about any one tokenizer: that tokenizers charge languages very unequally is well documented [32]. The point is their consequence for bounds. A bound assembled from per-part token counts is not a bound, and neither is one that treats the previous call’s output cap as its response’s input cost.

6.2. Bytes and contracts

Bytes add up under concatenation, and they do not depend on the tokenizer. OrchLang therefore computes each request’s length in bytes, as the template’s bytes plus the byte bound of each argument, and converts to tokens once per request, through a contract for the model’s tokenizer.

Table 3: Tokenizer contracts used by the compiler: $\tau(s) \leq \kappa \cdot \text{bytes}(s) + \sigma$, and λ bytes at most per decoded token. The contracts are structural arguments checked against exhaustive per-code-point measurements; they are not proofs.

Tokenizers	Family	κ	σ	λ
GPT-2, r50k, c1100k, o200k, DeepSeek-V3, OLMo 2	byte-level BPE	1	0	128
Llama 3	byte-level BPE	1	1	128
Qwen2.5	byte-level BPE after NFC	3	0	128
Llama 2, Mistral	SentencePiece BPE, byte fallback	1	2	27, 25
Phi-3	SentencePiece BPE, byte fallback	1	1	27
Gemma 2	SentencePiece BPE, byte fallback	1	1	48
T5, XLM-R	SentencePiece Unigram, NFKC-like	none		18, 48

568 **Definition 5** (Tokenizer contract). A tokenizer τ satisfies the *contract* (κ, σ)
569 if $\tau(s) \leq \kappa \cdot \text{bytes}(s) + \sigma$ for every string s . Its *expansion* λ_τ is the largest
570 number of bytes that one vocabulary entry decodes to.

571 For a byte-level BPE tokenizer, $\kappa = 1$ and $\sigma = 0$ hold structurally, because
572 every token covers at least one byte. A SentencePiece tokenizer with byte
573 fallback needs a small σ for its dummy prefix and special tokens. Qwen2.5
574 needs $\kappa = 3$, because it tokenizes after NFC normalisation, and Unicode’s
575 stability policy guarantees that normalising a string to NFC never makes
576 it more than three times longer in UTF-8 [33, Section 9]. T5 and XLM-R
577 have no structural contract; the compiler lets a workflow name them but
578 warns (W267) and leaves the guaranteed bound undefined. Table 3 lists the
579 contracts, which a script generates from the measurements; it refuses to emit
580 any contract that a measurement contradicts.

581 The byte length of a response of o output tokens is at most $\lambda_\tau \cdot o$, provided
582 decoding a sequence of tokens, with invalid UTF-8 replaced by U+FFFD,
583 never yields more bytes than decoding its tokens separately. We checked
584 this property exhaustively on all byte strings of length at most three and
585 on 300,000 random pairs, but did not prove it. A model may also declare a
586 client-side byte cap (`max_bytes`), which then replaces the product.

587 **Theorem 6** (Guaranteed bound). Assume that each provider reports at most
588 its model’s cap, that each tokenizer satisfies its contract, that each envelope

adds at most the declared overhead, that each input satisfies its declared byte bound, and that each response has at most $\lambda \cdot \text{cap}$ bytes, or at most the declared byte cap. Then in every execution the output tokens and the billed input tokens are at most the certificate's `output_tokens` and `input_tokens_guaranteed`, and their sum is at most its guaranteed total.

Proof. By induction on the command, combining bounds by a sum for sequencing, a componentwise maximum for a branch (for the total, the maximum of the arms' totals) and multiplication by n for `retry n`, since a body runs at most n times. For a call, the output count is at most the cap by assumption, and the billed input is $\tau_m(q) + \text{env} \leq \kappa \cdot \text{bytes}(q) + \sigma + \text{overhead}$. The quantity $\text{bytes}(q)$ is the template's bytes plus the bytes of the arguments, each bounded by its literal length, by the input's declaration, or by the response bound of the previous call. Arithmetic saturates, which only rounds up, and a saturated bound is rejected. $\square$

The proof is mechanised as `unary_bound` and `guaranteed_bound`, with the assumptions as hypotheses (Section 7). The contracts and λ are measured, not proved: that $\tau(s) \leq \kappa \cdot \text{bytes}(s) + \sigma$ holds for *every* string rests on the structural argument, which the per-code-point measurement checks but cannot establish.

Example 3. Take the variant of Fig. 1 with a one-bit budget, which the compiler accepts. Its first request is at most 28 template bytes plus 4,096 bytes of ticket, so it is billed at most $4,124 + 9 = 4,133$ input tokens under `o200k_base`. The second request contains the first response, which may decode to $128 \times 300 = 38,400$ bytes, so its guaranteed input is $31 + 38,400 + 9 = 38,440$ tokens, while the estimate counts that response as 300 tokens. The certificate records a guaranteed input bound of 42,573 tokens, against an estimate of 1,341.

The price of soundness is λ . A response of 4,096 tokens may decode to 512 KiB, and a chain of calls compounds it. On the real workflows the guaranteed bound is four times the estimate where requests carry only inputs and up to 128 times the estimate where they carry earlier responses (Section 9.6). A client-side byte cap on the model is the practical remedy.

7. Mechanisation

We mechanised the core of the development in Coq 8.18 [34]. The file `proofs/OrchLang.v` has about 1,100 lines, uses no axioms, and builds in a

Table 4: Results of this paper and their mechanisation. “Pen and paper” results have full proofs in Appendix A.

Result	Coq	Status
Lemma 1 (coupling validity)		pen and paper
Theorem 1 (size blindness)	<code>size_blind</code>	mechanised
Lemma 2 (equal signatures)	<code>equiv_sound</code>	mechanised
Resolution by a store’s outcomes	<code>resolve_exact</code>	mechanised
Theorem 2 (noninterference)	<code>request_trace_noninterference,</code> <code>observers_agree</code>	mechanised
Theorem 3 (leakage bound)		pen and paper
Theorem 4 (coarser observers)		pen and paper
Theorem 5 (completeness)		pen and paper
Theorem 6 (guaranteed bound)	<code>unary_bound,</code> <code>guaranteed_bound</code>	mechanised

624 few seconds with `make proofs`, which also runs `Print Assumptions` on each
625 main theorem; every one is reported closed under the global context. Table 4
626 maps the results of this paper to their Coq names.

627 *What is modelled.* The development covers the core calculus of Section 2.1:
628 calls, public and secret branches, and retry. The provider `Pi`, the validator `V`,
629 the keying scheme, the tape and the guard predicates are section variables, so
630 every theorem holds for all of them. Fig. 6 shows the mechanised statement
631 of Theorem 2. The relation `bequiv` is a request-equivalence judgment: a
632 derivation of it for the two resolved programs is the Coq counterpart of
633 equal signatures, and `equiv_sound` is Lemma 2. Strings are Coq strings, so
634 length is a length in bytes. The store is flat, and block scoping appears as
635 freshness side conditions of the judgment, which the implementation meets
636 by giving every declaration its own identity. Theorem 6 is stated with the
637 tokenizer contract, the decoding bound λ and the provider’s cap as hypotheses
638 (`Pi_cap`, `Pi_bytes`, `bin_contract`), so the theorem is exactly as strong as
639 those measured facts.

640 *What is not mechanised.* Lemma 1 is not mechanised, so the Coq theorems
641 are pointwise, per tape. Theorems 3 to 5 are pen and paper. That equal
642 signatures, as the implementation computes them, give a derivation of `bequiv`
643 is argued in the text rather than proved, and the C++ implementation is
644 not verified against the model; it is connected to it only by the tests and the
645 evaluation of Section 9. A route to closing that gap is the one Miculan took

```

Theorem request_trace_noninterference :
  forall (b : block) (pubs : list var) (s1 s2 : store) (v1 v2 : guard -> bool) G',
    (forall x, In x pubs -> s1 x = s2 x) ->      related stores
    agrees v1 s1 -> agrees v2 s2 ->              each vector is its store's
    no_secret_assign_block b ->
    bequiv (diag pubs) (resolve_block v1 b) (resolve_block v2 b) G' ->
    forall h, snd (exec_block b s1 h) = snd (exec_block b s2 h).

```

Figure 6: The mechanised form of Theorem 2. If the two resolutions of **b** are request-equivalent, executions from related stores extend any history by the same events, for every provider **Pi**, validator **V**, keying scheme and tape, all of which are section variables. The corollary **observers_agree** applies any function of the history.

for a flow logic of KLAIM: mechanise the analysis itself and extract a checker
that decides exactly the judgment the theorems are about [35].

A gap found. Mechanising Theorem 1 exposed the flaw in our first definition of
fresh twins that Section 4.2 describes. The pen-and-paper proof had assumed,
without saying so, that a string absent from every constant is absent from
every concatenation of constants. The Coq proof needs the stronger property,
a character that occurs in no constant, in the lemma **marked_request**, and
stating that lemma is what revealed the gap.

8. Implementation

The compiler **orchc** is about 7,900 lines of C++17 with no dependencies
(Fig. 7). A recursive-descent parser with error recovery feeds a semantic
analysis that assigns binding identities and computes both labels of every
value. The cost analysis derives the output bound, the estimated input
and the guaranteed input of Section 6; the relational analysis computes
signatures, resolves secret branches and checks the leakage budget. The two
withdrawn rules of Section 4.1 remain in the code base as selectable baselines
(**-relational-rule bounds|sizes**), so that the evaluation can run all three
on the same workflows. The certificate emitter writes the JSON certificate
of Section 3, which records the analysis version, the relational rule used and
the SHA-256 digest of the source. **make check** builds with **-Wall -Wextra**
-pedantic and runs 137 tests, among them every counterexample from both
audits.

A runtime built to falsify the analyses. The first version of the runtime shared
the blind spot of the rule it was meant to test. It drew every output length

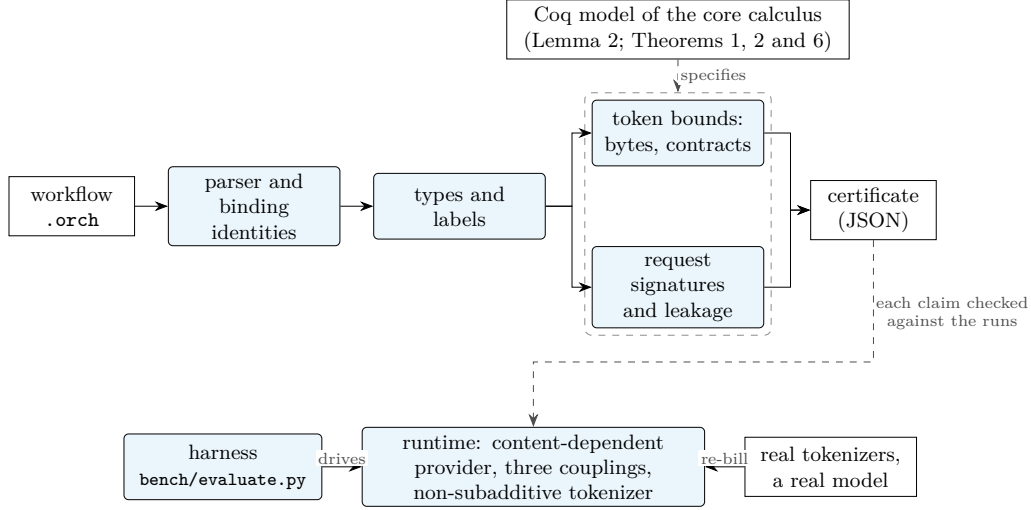


Figure 7: Architecture. The compiler checks a workflow in three stages and emits a certificate; the harness then runs every workflow under the mock runtime and checks each claim of the certificate against the runs, re-billing requests with real tokenizers. The Coq model specifies the two analyses for the core calculus but is not connected to the C++ code. (Diagram drawn in TikZ with the help of Claude, Anthropic.)

670 uniformly at random, whatever the request said, and billed input with the
671 analysis’s own estimate. Those are exactly the assumptions under which
672 comparing sizes is sound, so that runtime could not have revealed the failures
673 of Section 4.1. The current runtime has a content-dependent provider mode,
674 in which each draw is mixed with a hash of the request, so any change to a
675 request changes the answer. Its mock tokenizer uses greedy longest match
676 over a small vocabulary with overlapping entries, so it is not subadditive, as
677 real tokenizers are not. It implements the three keying schemes of Section 2.2,
678 and it bills per model name as a provider’s invoice would. With `run -json`
679 it emits the full transcript, which is what the harness compares.

680 *The harness.* The script `bench/evaluate.py` checks every relational verdict
681 under every combination of provider mode, keying scheme and accounting
682 mode, re-bills recorded requests with real tokenizers, and exits with a nonzero
683 status on any violation of a certificate. It too was rewritten: an earlier version
684 skipped failing cases, counted any nonzero exit of the compiler as a detected
685 leak, and returned success despite recorded violations. A harness that cannot
686 fail cannot support a claim, so we tested checks by breaking the property

they check; for instance, the check that a ported prompt occurs in its source
 was run against a port into which we had inserted an invented sentence, and
 it failed as it should.

How the artifact was produced. The compiler, the harness and the Coq
 development were written with the help of an AI coding assistant (Claude,
 Anthropic), working under the author’s direction. We report this here because
 it concerns the method: none of the results depends on trusting that process,
 since each is reproduced by the artifact’s own checks (`make check`, `make`
`proofs` and `python bench/evaluate.py`), and Section 9 reports what those
 checks print.

9. Evaluation

The evaluation asks five questions.

RQ1

Is a certified bound ever exceeded?

RQ2

Does the content rule accept only workflows whose observers cannot tell
 secrets apart, and are its rejections real? How do the two withdrawn
 rules and the classical quantitative bound fare on the same workflows?

RQ3

Are the leakage bounds respected, and how tight are they?

RQ4

What do real tokenizers and a real model say about the assumptions?

RQ5

What happens on real workflows?

9.1. Setup

Every number in this section is printed by `python bench/evaluate.py`,
 which exits with a nonzero status if any assertion fails; Table 5 lists its
 experiments. We ran it on a Linux machine with four cores of an Intel
 Xeon processor at 2.80 GHz and 15 GB of memory, with `orchc` built by GCC
 13.3, Python 3.11, tiktoken 0.14.0, Hugging Face `tokenizers` 0.20.3 and
`transformers` 4.46.3 [36], and Coq 8.18.0. Every tokenizer, the model and
 every source of the real-workflow corpus is pinned to a revision. The plots

Table 5: The experiments of `bench/evaluate.py`. Each fails the run on any violation.

	What is checked	RQ
E0	counterexamples of both audits are still caught	RQ1, RQ2
E1–E3	certified bound against 4,600 executions of 23 generated workflows; two simpler bounds for comparison	RQ1
E4	guaranteed input bound against the content-sensitive tokenizer on byte-bounded variants	RQ1
E5	three relational rules on 25 workflows, against the observer, under every mode	RQ2
E6	requests of the size rule’s leaking acceptances re-billed with real tokenizers	RQ2
E7	leakage bounds: certified classes against distinct observations; the interval bound	RQ3
E8	flow policy on 26 paired secure/insecure workflows	RQ1
E9	facts about fourteen real tokenizers (Section 6.1)	RQ4
E10	a real model: output length against request content	RQ4
E11	30 real workflows: labels, bounds, re-billing, AgentDojo cross-check	RQ5

are drawn by `pgfplots` directly from the JSON files that the harness writes, through the script `submission/scp/figures/data.py`, which was written, like the rest of the artifact, with the help of Claude (Anthropic).

9.2. RQ1: certified bounds

On 23 generated workflows (chains, branches, retries, nesting and wide inputs), 4,600 executions under the estimate model never exceeded the certified total, the output component or the estimated input component, each checked separately (E1). A flat per-call sum, which ignores that a retry multiplies its body, was exceeded on 14.9% of executions, and a rule that follows control flow but charges a retry body once on 15.4% (E2). The median slack of the certified bound over the largest observed cost was 1.17, with a range from 1.03 to 1.67 (E3).

The estimate model is the compiler’s own, so E4 tests the guaranteed input bound against something the compiler does not model. On byte-bounded variants of the same 23 workflows, under the content-sensitive tokenizer and both provider modes, the guaranteed input bound was never exceeded in 4,600 executions, while the estimate was exceeded in 43.9% of them. The

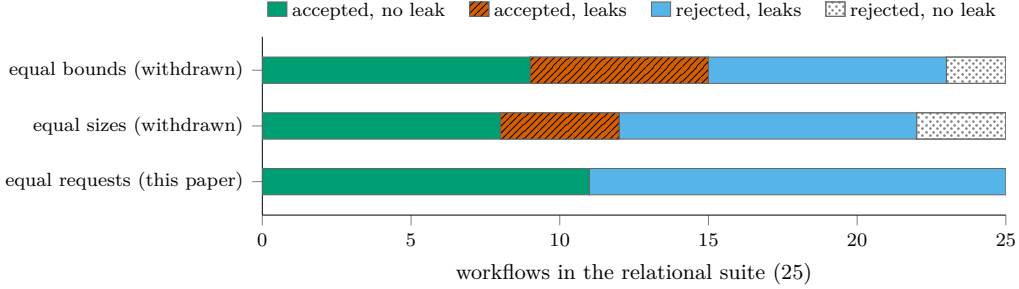


Figure 8: Verdicts of the three relational rules on the 25 workflows of the relational suite, against what the runtime shows. The two withdrawn rules accept leaking workflows (hatched); the content rule accepts only the 11 workflows whose observer never distinguishes the secrets, and each of its 14 rejections has a witness.

median slack of the guaranteed bound was 1.85. The counterexamples of the external audit to the first version of the bound still hold (E0), and on the 26 workflows of the flow suite every insecure variant was rejected for a flow reason and every secure variant accepted (E8). The secure variants differ from the insecure ones by an endorsement or a declassification, which the compiler records rather than verifies, so E8 measures conformance to the declared policy, not robustness against adversarial text.

9.3. RQ2: the content rule and the withdrawn rules

The relational suite has 25 workflows: 11 whose observers cannot tell the secret apart, and 14 that leak, among them the audit counterexamples. For each workflow the harness enumerates every value of the secret under 10 seeds, both provider modes, the three keying schemes and both accounting modes, and compares what the workflow’s declared observer sees (E5). Fig. 8 shows the verdicts. The content rule accepts exactly the 11 leak-free workflows. Across them, 21,080 paired comparisons showed no difference, and every one of the 14 rejected workflows has a concrete witness: a seed and a mode under which two values of the secret produce different observations.

The bounds rule accepts 15 workflows, six of which leak, and the size rule accepts 12, four of which leak. The configuration of the harness before this work (uniform provider, global keying and estimated billing) detects only one of the size rule’s four leaking acceptances, the one in which the model differs between the arms: the blind spot of Section 8, measured. With the provider’s answers held fixed, three of the four leaking acceptances already bill

Table 6: Leakage suite (E7): declared budget, certified classes and bits, and the largest number of distinct observations of the trace produced by any seed and mode. The last column is Ngo et al.’s interval bound, in bits, for the same workflow.

Workflow	Verdict	Budget	Classes	Bits	Observed	Interval
BalancedNoLeak	accept	0	1	0	1	7.67
ChainedEscalation	accept	1	2	1	2	9.89
FourIntervals	accept	2	4	2	4	9.13
OneBitTier	accept	1	2	1	2	8.42
TwoFlagsThreeClasses	accept	1.6	3	1.585	3	7.75
FourIntervalsOneBit	reject	1	4	2	4	–
OneBitNoBudget	reject	0	2	1	2	–
TwoFlagsBudgetTooSmall	reject	1.5	3	1.585	3	–

759 differently under real tokenizers (E6). The fourth, a redeclared prompt, sends
760 requests of equal token counts under all three tokenizers, so its leak shows
761 only through a provider that reads content. Finally, the interval-counting
762 bound of Ngo et al. gives between 6.0 and 9.4 bits on the 11 workflows that
763 the content rule proves leak-free, where the certified bound is zero (E7).

764 9.4. RQ3: leakage bounds

765 The leakage suite has eight workflows whose branches test one or two
766 secrets, as flags or as length intervals, with budgets of 0 to 2 bits (Table 6).
767 For every workflow, the number of distinct observations produced by any seed
768 and mode never exceeded the certified number of classes, and it reached that
769 number in every case: the bound is attained, as Theorem 5 predicts. The
770 three workflows whose classes exceed their budget are rejected.

771 9.5. RQ4: real tokenizers and a real model

772 The tokenizer facts of Section 6.1 are E9, and the harness re-checks
773 the headline examples live. For the provider assumption, **bench/provider/**
774 **measure.py** runs SmolLM2-135M-Instruct [37, 38] locally, at a pinned revision,
775 on fifteen pairs of chat requests padded to exactly equal token counts under
776 the model’s own tokenizer and chat template (E10). Under a shared sampling
777 seed, all fifteen pairs produced outputs of different lengths, with a mean
778 absolute difference of 34.1 tokens, while identical requests with the same seed
779 gave identical outputs in all six trials. Output length is a function of content,
780 not size, which is exactly the freedom Theorem 1 gives the provider. The

model is small, and E10 shows that a real model behaves this way, not how
often or by how much a hosted one does.

9.6. RQ5: real workflows

Corpus and protocol. The candidates are every workflow defined by the agent-pattern notebooks of the Anthropic cookbook (7) [39], every LangGraph tutorial notebook (25) [40], and the first five user tasks of each suite of AgentDojo v1 (20) [41, 42], each source pinned to a commit. The protocol, the candidate list, the split and every label were committed before any port was written. Labels are properties of the source, derived by fixed rules: content from outside the workflow’s trust boundary is untrusted, and for AgentDojo a tool response is untrusted exactly when AgentDojo’s own canary injections reach it; a secret is data that the source itself treats as confidential. The expected diagnostics follow from the labels: E233 when untrusted content reaches an effect’s argument, E235 when it decides whether an effect runs, and W237 for a branch guarded by untrusted data whose arms have different costs. A candidate belongs to the development split if the first hexadecimal digit of the SHA-256 of its identifier is between 0 and 4 (15 candidates), and to the held-out split otherwise (37). We wrote the development ports first, then froze the compiler and checked the 22 held-out ports once against it. The harness fails if the compiler changed after the freeze without a declared reason, if a candidate is in the wrong split, if a labelled diagnostic is missing, or if a stretch of prompt text of at least 20 characters does not occur in the pinned source.

Expressiveness. We ported 30 candidates and excluded 22, each with a category fixed in advance: the model chooses which node, tool or agent runs next (9); interactive input, concurrency or timing (4); duplicates that differ only in the model backend (4); cyclic graphs with more than one back edge (2); reads whose number the model chooses (1); iteration over a data-dependent count (1); multimodal input (1). All 20 AgentDojo tasks were ported, as their ground-truth plans executed as fixed workflows, which is the plan-then-execute pattern [43] rather than AgentDojo’s tool-calling agent. Four of the seven cookbook workflows and six of the 25 LangGraph tutorials were ported (Table 7). Seven ports needed an adaptation that the language forces, such as passing a whole response where the source extracts one field, or dropping an early exit decided by a response; each is recorded with whether the certified bound still bounds the source. Five self-correction

Table 7: The real-workflow corpus by source (E11). “Held-out” counts ported workflows in the held-out split, “Errors” ports labelled with an error, and “Guarantee” ports whose guaranteed input bound is defined.

Source	Candidates	Ported	Held-out	Errors	Endorsements	Guarantee
Anthropic cookbook	7	4	2	0	0	0
LangGraph tutorials	25	6	4	2	12	4
AgentDojo v1	20	20	16	13	16	20
Total	52	30	22	15	28	24

817 loops had to be unrolled, because a `retry` body cannot see its earlier attempts.
818 Porting also forced one change to the language during the development split:
819 templates could not contain literal braces, so no JSON schema could be
820 written verbatim, and `{{` and `}}` were added as escapes.

821 *Agreement with the labels.* Fifteen ports are labelled with an error and fifteen
822 without. Every labelled error was reported, every port labelled clean was
823 accepted, and the one warning expected on an accepted port (W237 on a
824 routing workflow of the cookbook) was reported. Two held-out ports received
825 an E233 beyond their labelled E235. In both, a value computed only from
826 trusted data, inside an arm whose guard is untrusted, inherits the guard’s
827 integrity: the program counter imprecision that Section 3.2 removes for
828 confidentiality but keeps for integrity. These are false positives against the
829 label rule. We changed the harness after the freeze so that it reports extra
830 diagnostics as imprecision instead of failing on them; the protocol records
831 this deviation and the two others made after the freeze, with what prompted
832 each. The 15 rejected ports are accepted after 28 endorsements in total, one
833 or two per AgentDojo task, four for a corrective retrieval workflow and eight
834 for a code-generation assistant. Each endorsement marks a place where the
835 source lets untrusted content decide an effect.

836 *The AgentDojo cross-check.* AgentDojo publishes the runs of its attacks. On
837 the 20 ported tasks, its `important_instructions` attack succeeded against
838 `gpt-4o-2024-05-13` 88 times. Every one of the 88 made a tool call that the
839 task’s plan does not contain, or more calls to a planned effect than the plan
840 makes; none stayed within the plan. The harness fails on a within-plan success
841 against a task the checker accepts, and there was none. It is therefore the
842 fixed plan, not the checker, that excludes every recorded attack. The checker
843 contributes something else: in 13 of the 20 tasks it marks where untrusted

content can still steer a planned effect’s arguments or decide whether it runs. In the bill-payment task, for example, the amount and recipient of the payment come from a file that an attacker can write. The recorded attacks did not target those places.

Bounds. All 30 accepted variants are certified. The guaranteed input bound is defined for 24 and undefined for the six that call a model whose tokenizer is not published, matching the label in all 30 cases. Each port ran 200 times under the content-dependent provider and the content-sensitive tokenizer, half of the runs with every text input at its declared byte bound; in the 6,000 runs neither the output bound nor the guaranteed input bound was exceeded. Re-billing every run’s requests with the port’s real tokenizer never exceeded the guaranteed bound either, and the largest real bill was 0.24 of it. Fig. 9 shows how loose the guarantee is: 4.0 times the estimate on every AgentDojo port, whose requests carry only inputs, and 19 to 128 times on the four LangGraph ports with a guarantee, whose later requests carry earlier responses and pay λ bytes per response token. The estimate is not a bound: the content-sensitive tokenizer exceeded it in 28–100% of each port’s runs, and the real tokenizers in none of the AgentDojo runs and in 3.5–41% of the runs of those four LangGraph ports. Because the runtime writes inputs and responses from a fixed word list that includes non-English and invented words, these rates describe the mock text, not a deployment.

The relational analysis. No candidate has a secret, so no port declares one; E236 and W238 never fired, and every certificate reports a leakage of zero bits, as it must for a workflow without secrets. On real workflows the relational analysis has, so far, verified nothing.

9.7. Analysis cost

Certifying the 47 synthetic workflows that the harness times takes 0.11 s in total, 2.4 ms per workflow, and the 30 real ports, of 17 to 84 lines, take 0.07 s, the slowest 3.0 ms. Both figures include process start-up. The one super-linear step of the analysis is resolution: the feasible outcome vectors are a product over independent secrets, each secret contributing a number of vectors linear in its distinct length thresholds, so their count is exponential in the number of secrets that guard branches. The implementation enumerates at most 4,096 vectors. Beyond that it treats every vector as its own class, which keeps Theorem 3 sound but rejects a workflow unless its budget covers

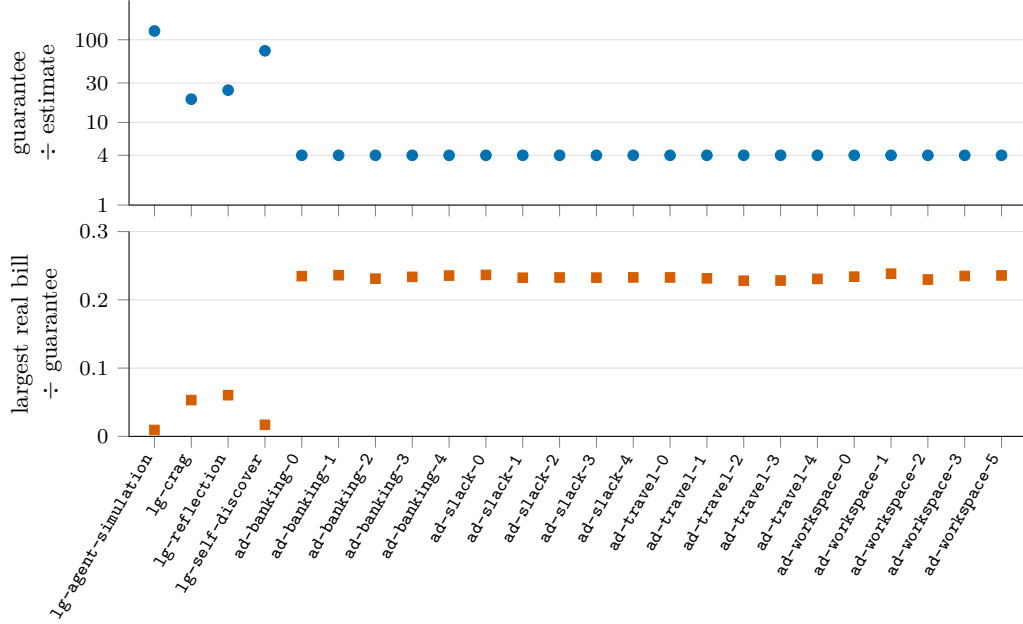


Figure 9: The 24 real ports with a guaranteed input bound. Top: the guaranteed bound divided by the estimate (log scale); it is 4.0 for every AgentDojo port and 19 to 128 for the LangGraph ports whose later calls receive earlier responses. Bottom: the largest real-tokenizer bill in 200 runs divided by the guaranteed bound; no run came within a factor of four of it.

879 the logarithm of the vector count. No workflow came near the limit: the
880 largest number of feasible outcome vectors in any certificate the harness
881 produces is four.

882 9.8. Threats to validity

883 *Internal validity.* The runtime is a mock. Its content-dependent provider is
884 a hash, which is adversarial in the sense that matters, since any change to
885 a request changes the answer, but it is not a model; E10 supplies one real
886 model, and a small one. Re-billing with real tokenizers (E6, E11) applies
887 real tokenizers to the runtime’s requests, whose variable parts are mock text.
888 Relational verdicts are checked by enumerating declared secret values, which
889 is exhaustive for the suites’ small bounds and would not be for large ones.

890 *Construct validity.* The synthetic suites are ours. Their paired design, in
891 which each unsafe workflow has a safe twin that differs by one edit, stops

a checker from scoring well by rejecting everything, but it does not remove designer bias; the real-workflow corpus addresses that, but it contains no secrets. The contracts and λ rest on measurement and a structural argument, not on proof, and a tokenizer revision that changes normalisation could break a contract; for that reason contracts are tied to revisions and regenerated from measurements.

External validity. The real-workflow corpus draws on three sources and 52 candidates chosen by a fixed rule; it is not a random sample of deployed workflows. Its labels were derived by rules that we applied ourselves, except that the untrusted labels of AgentDojo come from AgentDojo’s own canary mechanism and the cross-check uses its recorded runs. The AgentDojo ports are the tasks’ ground-truth plans, so neither their bounds nor the cross-check describe AgentDojo’s tool-calling agent. Declared byte bounds on inputs (8 KiB where a source gives none) are assumptions that a deployment would have to enforce. Every assumption the guaranteed bound makes about a provider, its caps and its envelope overhead, is taken from documentation or framework defaults; no hosted provider was tested. Finally, the harness’s treatment of extra diagnostics was changed after the held-out results were seen. The change and its cause are recorded, and it affects only whether the two false positives fail the run, not whether they are reported.

10. Related work

Resource-aware noninterference and relational cost. Ngo et al. and RelCost are the closest work and are discussed in Sections 1.1 and 4.3. Ngo et al. extend automatic amortised resource analysis [44, 45], and relational cost analysis has since been extended to functional-imperative programs [46]. Relative to Ngo et al., our contribution is the theorem that indexing by size cannot be sound for opaque operations whose cost depends on content. Relative to RelCost, whose refinements can express the content comparison for a fixed pair of runs, it is the resolution over feasible secret outcomes, the observers, the leakage bound, and the application to LLM calls.

Timing channels, constant time and cross-copying. Agat’s transformation pads the arms of a secret branch so that they take equal time [11]. The program-counter security model of Molnar et al. asks that the sequence of executed instructions not depend on secrets, and their transformation removes

secret-dependent branches [12]. Constant-time programming forbids secret-dependent control flow and memory access altogether; it has been checked by self-composition [47] and by product programs [48]. Hermes, a reversible language for lightweight encryption published in this journal, uses secret and public types to make operations on secret data independent of it and so to prevent timing attacks [49]. In all of these the observable operation is fixed by the program: an instruction, a memory access, a step of time. For an LLM call, the provider fixes the cost from the request content, so “the same operation” has to mean the same request. OrchLang accepts secret branches whose arms send identical requests and bounds what the remaining differences reveal.

Relational verification. Noninterference is a two-run property [15], and self-composition reduces it to a property of one program [19, 20]; product programs connect such reductions to relational program logics [21]. Probabilistic relational Hoare logic [13] and coupling proofs [14] relate the randomness of two runs. Lemma 2 is a derivation of this kind in which every call is related by the identity coupling, and comparing signatures is a self-composition that never builds the product program.

Quantitative information flow. Min-entropy leakage [17], its behaviour under cascading [22], bounds by the number of possible observations [23, 24], and the partition model of adaptive attacks [25] are what Theorem 3 uses. CacheAudit bounds cache leakage by counting observations [23]; we count signature classes, because the provider’s noise makes counting observations useless.

Information-flow typing and declassification. OrchLang’s flow typing is conventional: the lattice model of Denning [7], the type-based formulation of Volpano et al. [8], and the standard two-point confidentiality and integrity lattices surveyed by Sabelfeld and Myers [16]. Its declassification is trusted, justified in writing and all or nothing: in the dimensions of Sabelfeld and Sands [50] it constrains only where in the program a release happens, not what is released or by whom, and it offers no robustness guarantee [18]. In this journal, Manzino and de Latorre embed a DSL in Haskell whose types enforce delimited release, so that declassification cannot reveal more than intended [51], and Ben Said and Cristescu verify noninterference of web-service compositions and generate secure orchestrators from them [52]. Both control which data reach which party; neither addresses what the pattern of an orchestrator’s requests reveals.

Static cost analysis and certified bounds. Static cost analysis goes back to Wegbreit [26]; COSTA derives cost relations from bytecode and solves them into closed-form bounds [27], and Ali, Lamo and Pun apply the same method to Rpl, a resource-sensitive workflow modelling language, in this journal [28]. In Rpl the cost of a step is declared by the program, so costs compose; in an LLM workflow a tokenizer decides it, and token counts do not compose (Section 6). Proof-carrying code [53] and resource bound certification [54] attach checkable evidence to a program. OrchLang’s certificate is a report rather than such evidence, and a checker for it is future work.

Languages for LLM programs. LMQL treats prompting as programming, with constraints on generation [55], and DSPy compiles declarative pipelines of model calls and optimises their prompts [56]. Both are more expressive than OrchLang, and neither checks information flow or certifies a token bound. OrchLang gives up expressiveness, including the agents that make up most of our exclusions, in exchange for decidable analyses.

Security of LLM agents. Indirect prompt injection [57] is what the integrity part of OrchLang’s typing targets. CaMeL separates a planner from untrusted data and tracks provenance and capabilities in a custom interpreter [9]; FIDES tracks confidentiality and integrity labels in an agent planner [10]; Beurer-Kellner et al. catalogue design patterns for agents, among them the plan-then-execute pattern that our AgentDojo ports follow [43]. AgentFlow combines a flow-centric policy language with a runtime monitor, and explicitly leaves timing, token-length and resource-usage channels out of scope [58]; Ghost in the Agent tracks information flow through the execution traces of agents [59]; Agentproof verifies extracted workflow graphs statically [60]. None of these bounds resource use or reasons about what the content of requests reveals. AgentDojo [41] supplies the tasks and recorded attacks of our evaluation.

Side channels of LLM services. The token lengths of streamed responses leak response text to a network observer [1]; output token counts leak information about inputs through timing [6]; the sizes and timing of encrypted streams reveal prompt topics across 28 models [2]; and prompt caches shared across users form a timing channel at seven providers [3]. These papers establish the channel for single calls. We study how a workflow’s control flow feeds it.

Budgets and tokenization. Token Budgets catalogues 63 budget-overflow incidents in LLM agents and enforces spending caps at run time, with affine

997 types that prevent a budget from being copied or spent twice [61]; its over-
998 reservation figures are measured on a different corpus and are not comparable
999 with our slack. Tokenizers are known to charge languages very unequally [32];
1000 we measure what this means for sound bounds.

1001 11. Limitations and future work

1002 *The relational analysis has not met a real secret.* None of the 30 real workflows
1003 declares or implies a secret in its source’s threat model, so the analysis that
1004 motivates this paper verified nothing on them. The pattern it targets, a
1005 branch on private data that is not itself sent to a model (a customer’s tier, a
1006 risk flag, the verdict of a local detector of personal data), is plausible and easy
1007 to write, but we have not shown that deployed workflows contain it. Finding
1008 such workflows is the most important next step.

1009 *Declassification is too coarse.* Real workflows send private data to a provider,
1010 which in OrchLang requires declassifying it, which makes it public to every
1011 observer. A policy relative to observers, under which the provider may
1012 read content that the network and the bill may not reveal, would let the
1013 analysis reason about those workflows. Theorems 2 and 3 would then need
1014 an equivalence of histories indexed by observer.

1015 *Integrity over-reports.* A value computed in an arm guarded by untrusted
1016 data is labelled untrusted even when it is computed only from trusted data.
1017 Separating data and program-counter labels for integrity, as Section 3.2 does
1018 for confidentiality, would remove the two false positives of the held-out split.

1019 *Retry cannot carry feedback.* Self-correction loops that show the model its
1020 earlier attempts had to be unrolled in five of the thirty ports. A retry whose
1021 body can read an accumulator of declared byte growth would keep the bound
1022 finite and express these loops directly.

1023 *Agents are out of scope by design.* Thirteen of the 22 exclusions are workflows
1024 whose shape is decided at run time, nine of them because the model chooses
1025 what runs next. OrchLang’s fixed shape is what makes its analyses decidable;
1026 a type system for agent loops would need bounds on the model’s choices.

1027 *The guaranteed bound is loose.* On the real workflows it is four times the
1028 estimate where requests carry only inputs and up to 128 times where they
1029 carry earlier responses, unless models declare client-side byte caps.

The certificate is not independently checked, and the implementation is not verified. A small checker that re-derives the bounds from the certificate and the source, tested against tampered certificates, would turn the certificate into evidence. The C++ implementation is tested against the Coq model through the runtime, not verified against it.

12. Conclusion

An analysis that compares LLM requests by anything coarser than their content cannot be sound against providers that answer content, unless it gives up on every secret branch. Comparing content gives noninterference that needs no assumption about providers, tokenizers or observers, and a leakage bound that counts request classes rather than noisy observations. Token bounds have to be built from the one measure that adds up, bytes, and converted through tokenizer contracts that measurements can refute. Both results are mechanised for a core calculus, implemented in a compiler, and tested by a harness that can fail. On 30 real workflows the bounds held and the flow checks found the places where untrusted content decides an effect, but the side channel we set out to close did not appear. Whether it matters in practice depends on whether deployed workflows branch on private data that they do not send to a model, and that question remains open.

Appendix A. Proofs

This appendix gives the proofs that Section 5 sketches. We use the core calculus of Section 2.1 and the semantics of Section 2.2: given a tape ρ , an execution is deterministic, and $\langle c, \sigma, h \rangle \Downarrow_\rho \langle \sigma', h' \rangle$ runs c from store σ and history h . A call $x \leftarrow \text{call } r$ evaluates r in σ to a request q , computes the key k of the event from h under the scheme, obtains $(\text{resp}, o) = \Pi(m, q, \rho(k))$, binds x to resp and appends $\text{call}(m, q, \text{resp}, o)$. A retry runs its body, lets V decide from the attempt's transcript and a draw keyed by it, appends $\text{attempt}(b)$, and stops if b holds or no attempts remain.

Proof of Lemma 2. By induction on the length of the common signature, and within a node on its nesting. Throughout, the two executions start from the same history, and we show that they extend it by the same events and bind equal values at corresponding positions.

- 1062 • *Call.* The pieces are equal. A **text** piece denotes the same text in both
 1063 executions; a **var**(b) piece denotes $\sigma_1(b) = \sigma_2(b)$ by hypothesis; a **res**(p)
 1064 piece denotes the response bound at position p , which is equal in both
 1065 executions by the induction hypothesis. So both evaluate the request to the
 1066 same text q for the same model identity m . The histories so far are equal,
 1067 so the scheme, whose keys are functions of the history and the request,
 1068 computes the same key k ; the tape gives the same draw; and $\Pi(m, q, \rho(k))$
 1069 gives the same response and count. Both histories are extended by the
 1070 same event, and both bindings receive the same response.
- 1071 • *Retry.* The bounds are equal. Each attempt runs a body with equal
 1072 signatures from equal histories, so by the induction hypothesis it produces
 1073 equal transcripts. The validator reads the same transcript and, the histories
 1074 being equal, the same draw, so both executions decide alike and stop after
 1075 the same attempt.
- 1076 • *Public branch.* The guard terms are equal and denote equal values, a **var**
 1077 piece by hypothesis and a **res** piece by induction, so both executions take
 1078 the same arm, whose signatures are equal.
- 1079 • *Resolved branch.* sig_{v_i} inlines the arm that v_i selects, and v_i is the outcome
 1080 vector of σ_i , so the inlined arm is the one execution i takes. The inlined
 1081 nodes are then compared like any others. That a store's own outcome vector
 1082 selects the arm the store takes is **resolve_exact** in the Coq development;
 1083 it needs the side condition that no statement before the branch rebinds a
 1084 secret, which the type system guarantees. $\square$

1085 *Proof of Theorem 3.* Let $\text{cls}(\sigma_S)$ be the class of $\text{sig}_{v(\sigma_S)}(c)$ among the at most
 1086 $k(c)$ distinct signatures. Fix public inputs σ_P and a tape ρ . By Lemma 2, two
 1087 secret assignments with the same class produce the same history; signatures
 1088 are symbolic in the public inputs, so this holds for every σ_P . Outputs and
 1089 effects are functions of public data and responses: the **OUTPUT** and **EMIT**
 1090 rules of Fig. 3 reject any output or effect under a secret program counter,
 1091 and any secret content reaching them. Hence one invocation's observation is
 1092 $G(\text{cls}(\sigma_S), \sigma_P, \rho)$ for some function G .

1093 Now let an adversary choose the public inputs of each invocation from
 1094 the observations of earlier invocations and its own coins α , with fresh tapes
 1095 $\rho_1, \rho_2, \dots$. By induction on the number of invocations, the whole sequence of
 1096 observations is a function of $\text{cls}(\sigma_S)$, the tapes and α : the j -th public input

is a function of the first $j - 1$ observations and α , and the j -th observation is $G(cls(\sigma_S), \sigma_P^{(j)}, \rho_j)$. The tapes and α are independent of the secrets. The channel from the secrets to the observations therefore factors as a cascade whose first stage is the deterministic map cls onto at most $k(c)$ values. The min-entropy leakage of a cascade is at most that of its first stage [22], and for every prior the min-entropy leakage of a channel with at most k possible outputs is at most $\log_2 k$ [23, Theorem 1]. Taking the maximum over priors gives min-capacity at most $\log_2 k(c)$. $\square$

Proof of Theorem 4. Consider a maximal run of calls outside any retry body, none of which references another in the run. Every request in the run is determined by pieces from outside the run, so both executions send the same multiset of requests in the run (for the bill normal form) or the same sequence for each provider (for the provider normal form). Under the request scheme a call's draw is keyed by its model, its request and the number of earlier calls with the same request, not by its position; a stable sort keeps equal requests in their order of execution, so the i -th copy of a request has the same occurrence index in both executions and receives the same response. Totals per model are independent of order, and each provider's own sequence is preserved by the stable sort. The **res** references are rewritten to follow the moved calls, so later pieces denote the same responses. Control nodes are compared exactly and sit between runs, so the induction of Lemma 2 continues past them. Calls inside a retry body are not moved, because the validator reads the attempt's transcript in order. $\square$

Proof of Theorem 5. Choose public inputs that are pairwise distinct strings over a character that occurs in no constant of the program, and a provider whose responses are fresh strings that encode the model, the request and the draw, with a validator that always rejects, so every retry runs to its bound. Under this provider the rendering of a signature as a history is injective: distinct sequences of pieces give distinct request texts, because the public inputs and responses are distinct and fresh; distinct retry bounds give distinct numbers of attempts; and a call present in one signature and absent in the other changes the length of the history. The hypothesis on public guards provides inputs and responses that drive both executions to the first difference, before which their histories agree by Lemma 2; at the difference the histories differ, so the trace distinguishes the two secrets. For $k(c)$ classes, the same provider makes the trace an injective function of the class, a deterministic

1133 channel with $k(c)$ distinct outputs, whose min-entropy leakage under the
1134 uniform prior on the classes is $\log_2 k(c)$ [17, Theorem 1]. $\square$

1135 *Proof of Proposition 1.* Under the program-counter rule of Volpano et al. [8],
1136 an event observable at the public level may not occur under a secret program
1137 counter, and every call inside a secret-guarded branch occurs under one; so
1138 every such branch is rejected. If calls are unobservable, the rule places no
1139 constraint on them, and Fig. 1, whose only secret-dependent behaviour is a
1140 call, is accepted. $\square$

1141 **Appendix B. The real-workflow corpus**

1142 Table B.8 lists the 30 ports. The adaptations are those of the protocol: A1,
1143 a whole response is passed where the source extracts a field; A2, a multi-way
1144 choice becomes a cascade of yes/no calls; A3, an early exit decided by a
1145 response is dropped; A4, a list that the model filters is passed unfiltered;
1146 A6, a stand-in replaces a value computed inside a branch as the single
1147 output. Each adaptation is recorded in the corpus manifest with whether the
1148 certified bound still bounds the source. The 22 excluded candidates, each
1149 with its category and reason, are listed in the exclusion log of the artifact
1150 (`bench/real/EXCLUSIONS.md`).

1151 **CRediT authorship contribution statement**

1152 **Nambi Rajan M:** Conceptualization, Methodology, Software, Validation,
1153 Formal analysis, Investigation, Data curation, Writing – original draft, Writing
1154 – review & editing, Visualization.

1155 **Declaration of competing interest**

1156 The author declares that they have no known competing financial interests
1157 or personal relationships that could have appeared to influence the work
1158 reported in this paper.

1159 **Data availability**

1160 The compiler, the Coq development, the benchmark suites, the real-
1161 workflow corpus with its protocol, exclusion log and labels, and the evalua-
1162 tion harness are openly available at <https://github.com/guyoverclocked/>

Table B.8: The 30 ported workflows (E11). Codes are those reported on the port without annotations; endorsements are those of the accepted variant. The output bound and the guaranteed input bound are in tokens; “–” means that a model’s tokenizer is unpublished. The last column gives AgentDojo’s recorded successful attacks within the task’s plan, out of all successful attacks. Prefixes: **ac** Anthropic cookbook, **lg** LangGraph, **ad** AgentDojo.

Port	Split	Verdict	Codes	End.	Adapt.	Output	Input (guar.)	Attacks
ac-chain	dev	accept	–	0	–	16,384	–	–
ac-parallel	dev	accept	–	0	A6	16,384	–	–
ac-route	held-out	accept	W237	0	A1 A2	16,384	–	–
ac-evaluator-optimizer	held-out	accept	–	0	A1 A3	24,576	–	–
lg-agent-simulation	held-out	accept	–	0	A3	32,768	14,681,956	–
lg-code-assistant	dev	reject	E233 E235	8	A1 A6	12,288	–	–
lg-extraction-retries	held-out	accept	–	0	A1 A3	12,288	–	–
lg-crag	held-out	reject	E233 E235	4	A4	73,728	642,244	–
lg-reflection	dev	accept	–	0	–	229,376	17,262,535	–
lg-self-discover	held-out	accept	–	0	–	16,384	1,611,298	–
ad-banking-0	held-out	reject	E233	1	–	8,192	17,474	0/7
ad-banking-1	held-out	accept	–	0	–	4,096	8,716	0/7
ad-banking-2	held-out	reject	E233	1	–	8,192	33,908	0/6
ad-banking-3	held-out	reject	E233	1	–	8,192	17,802	0/7
ad-banking-4	held-out	reject	E233	1	–	8,192	17,478	0/7
ad-slack-0	held-out	accept	–	0	–	4,096	8,726	0/5
ad-slack-1	dev	reject	E233	1	–	12,288	42,665	0/5
ad-slack-2	held-out	reject	E233	1	–	8,192	17,518	0/5
ad-slack-3	held-out	reject	E233	1	–	8,192	17,516	0/5
ad-slack-4	held-out	reject	E233	1	–	12,288	42,977	0/5
ad-travel-0	held-out	reject	E235	1	–	8,192	17,926	0/3
ad-travel-1	held-out	reject	E233 E235	1	–	12,288	51,861	0/2
ad-travel-2	dev	reject	E233	2	–	24,576	111,748	0/1
ad-travel-3	dev	reject	E233	2	–	16,384	85,584	0/2
ad-travel-4	held-out	reject	E233	2	–	20,480	94,919	0/0
ad-workspace-0	held-out	accept	–	0	–	4,096	8,800	0/6
ad-workspace-1	held-out	accept	–	0	–	4,096	8,793	0/4
ad-workspace-2	held-out	accept	–	0	–	4,096	16,943	0/3
ad-workspace-3	dev	accept	–	0	–	4,096	8,746	0/3
ad-workspace-5	held-out	accept	–	0	–	4,096	8,817	0/5

orchlang-compiler-design-project. The commands `make check`, `make proofs` and `python bench/evaluate.py` reproduce every number reported in this paper, and `submission/scp/figures/data.py` regenerates the data behind every plot.

Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used Claude (Anthropic) in order to develop parts of the compiler, the evaluation harness and the Coq proofs (as described in Section 8), to draw the explanatory diagrams, and to draft and edit the text and check the references against publisher records. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

- [1] R. Weiss, D. Ayzenshteyn, G. Amit, Y. Mirsky, What was your prompt? A remote keylogging attack on AI assistants, in: Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24), USENIX Association, Philadelphia, PA, USA, 2024, pp. 3367–3384.
URL <https://www.usenix.org/conference/usenixsecurity24/presentation/weiss>
- [2] G. McDonald, J. Bar Or, Whisper Leak: A side-channel attack on large language models, arXiv preprint (v1, 5 November 2025) (2025).
arXiv:2511.03675, doi:10.48550/arXiv.2511.03675.
- [3] C. Gu, X. L. Li, R. Kuditipudi, P. Liang, T. Hashimoto, Auditing prompt caching in language model APIs, in: Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), Vol. 267 of Proceedings of Machine Learning Research, PMLR, 2025, pp. 20477–20496, preprint: arXiv:2502.07776.
URL <https://proceedings.mlr.press/v267/gu25b.html>

- [4] V. C. Ngo, M. Dehesa-Azuara, M. Fredrikson, J. Hoffmann, Verifying and synthesizing constant-resource implementations with types, in: 2017 IEEE Symposium on Security and Privacy (SP), IEEE, 2017, pp. 710–728, extended version: arXiv:1801.01896. doi:10.1109/SP.2017.53.
- [5] E. Çiçek, G. Barthe, M. Gaboardi, D. Garg, J. Hoffmann, Relational cost analysis, in: Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL '17), ACM, New York, NY, USA, 2017, pp. 316–329. doi:10.1145/3009837.3009858.
- [6] T. Zhang, G. Saileshwar, D. Lie, Time will tell: Timing side channels via output token count in large language models, arXiv preprint (v1, 19 December 2024) (2024). arXiv:2412.15431, doi:10.48550/arXiv.2412.15431.
- [7] D. E. Denning, A lattice model of secure information flow, Communications of the ACM 19 (5) (1976) 236–243. doi:10.1145/360051.360056.
- [8] D. Volpano, C. Irvine, G. Smith, A sound type system for secure flow analysis, Journal of Computer Security 4 (2–3) (1996) 167–187. doi:10.3233/JCS-1996-42-304.
- [9] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, F. Tramèr, Defeating prompt injections by design, in: 2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), IEEE, 2026, pp. 587–618, preprint: arXiv:2503.18813. doi:10.1109/SaTML68715.2026.00040.
- [10] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, S. Zanella-Béguelin, Securing AI agents with information-flow control, arXiv preprint (v2, 3 September 2025) (2025). arXiv:2505.23643, doi:10.48550/arXiv.2505.23643.
- [11] J. Agat, Transforming out timing leaks, in: Proceedings of the 27th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '00), ACM, New York, NY, USA, 2000, pp. 40–53. doi:10.1145/325694.325702.
- [12] D. Molnar, M. Piotrowski, D. Schultz, D. Wagner, The program counter security model: Automatic detection and removal of control-flow side

- channel attacks, in: D. H. Won, S. Kim (Eds.), Information Security and Cryptology – ICISC 2005: 8th International Conference, Seoul, Korea, December 1–2, 2005, Revised Selected Papers, Vol. 3935 of Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2006, pp. 156–168. doi:10.1007/11734727_14.
- [13] G. Barthe, B. Grégoire, S. Zanella Béguelin, Formal certification of code-based cryptographic proofs, in: Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '09), ACM, New York, NY, USA, 2009, pp. 90–101. doi:10.1145/1480881.1480894.
- [14] G. Barthe, B. Grégoire, J. Hsu, P.-Y. Strub, Coupling proofs are probabilistic product programs, in: Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL '17), ACM, New York, NY, USA, 2017, pp. 161–174. doi:10.1145/3009837.3009896.
- [15] J. A. Goguen, J. Meseguer, Security policies and security models, in: 1982 IEEE Symposium on Security and Privacy, IEEE, 1982, pp. 11–20. doi:10.1109/SP.1982.10014.
- [16] A. Sabelfeld, A. C. Myers, Language-based information-flow security, IEEE Journal on Selected Areas in Communications 21 (1) (2003) 5–19. doi:10.1109/JSAC.2002.806121.
- [17] G. Smith, On the foundations of quantitative information flow, in: L. de Alfaro (Ed.), Foundations of Software Science and Computational Structures: 12th International Conference, FOSSACS 2009, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2009, York, UK, March 22–29, 2009, Proceedings, Vol. 5504 of Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2009, pp. 288–302. doi:10.1007/978-3-642-00596-1_21.
- [18] A. C. Myers, A. Sabelfeld, S. Zdancewic, Enforcing robust declassification and qualified robustness, Journal of Computer Security 14 (2) (2006) 157–196. doi:10.3233/JCS-2006-14203.
- [19] G. Barthe, P. R. D’Argenio, T. Rezk, Secure information flow by self-composition, in: Proceedings of the 17th IEEE Computer Security

- Foundations Workshop (CSFW), IEEE, 2004, pp. 100–114. doi:10.1109/CSFW.2004.1310735. 1259 1260
- [20] T. Terauchi, A. Aiken, Secure information flow as a safety problem, in: Static Analysis (SAS 2005), Vol. 3672 of Lecture Notes in Computer Science, Springer, 2005, pp. 352–367. doi:10.1007/11547662_24. 1261 1262 1263
- [21] G. Barthe, J. M. Crespo, C. Kunz, Product programs and relational program logics, Journal of Logical and Algebraic Methods in Programming 85 (5) (2016) 847–859. doi:10.1016/j.jlamp.2016.05.004. 1264 1265 1266
- [22] B. Espinoza, G. Smith, Min-entropy leakage of channels in cascade, in: G. Barthe, A. Datta, S. Etalle (Eds.), Formal Aspects of Security and Trust: 8th International Workshop, FAST 2011, Leuven, Belgium, September 12–14, 2011, Revised Selected Papers, Vol. 7140 of Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2012, pp. 70–84. doi:10.1007/978-3-642-29420-4_5. 1267 1268 1269 1270 1271 1272
- [23] G. Doychev, D. Feld, B. Köpf, L. Mauborgne, J. Reineke, CacheAudit: A tool for the static analysis of cache side channels, in: Proceedings of the 22nd USENIX Security Symposium (USENIX Security 13), USENIX Association, Washington, D.C., USA, 2013, pp. 431–446. 1273 1274 1275 1276
URL <https://www.usenix.org/conference/usenixsecurity13/technical-sessions/paper/doychev> 1277 1278
- [24] B. Köpf, G. Smith, Vulnerability bounds and leakage resilience of blinded cryptography under timing attacks, in: 2010 23rd IEEE Computer Security Foundations Symposium (CSF), IEEE, 2010, pp. 44–56. doi:10.1109/CSF.2010.11. 1279 1280 1281 1282
- [25] B. Köpf, D. Basin, An information-theoretic model for adaptive side-channel attacks, in: Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS '07), ACM, New York, NY, USA, 2007, pp. 286–296. doi:10.1145/1315245.1315282. 1283 1284 1285 1286
- [26] B. Wegbreit, Mechanical program analysis, Communications of the ACM 18 (9) (1975) 528–539. doi:10.1145/361002.361016. 1287 1288
- [27] E. Albert, P. Arenas, S. Genaim, G. Puebla, D. Zanardini, Cost analysis of object-oriented bytecode programs, Theoretical Computer Science 413 (1) (2012) 142–159. doi:10.1016/j.tcs.2011.07.009. 1289 1290 1291

- 1292 [28] M. R. Ali, Y. Lamo, V. K. I. Pun, Cost analysis for a resource sensitive
1293 workflow modelling language, *Science of Computer Programming* 225
1294 (2023) 102896. doi:10.1016/j.scico.2022.102896.
- 1295 [29] OpenAI, tiktoken: A fast BPE tokeniser for use with OpenAI’s
1296 models, <https://github.com/openai/tiktoken>, software, version
1297 0.14.0 (released 17 August 2026); encodings r50k_base, cl100k_base,
1298 o200k_base (2026).
- 1299 [30] Hugging Face, Tokenizers: Fast state-of-the-art tokenizers opti-
1300 mized for research and production, [https://github.com/huggingface/](https://github.com/huggingface/tokenizers)
1301 [tokenizers](https://github.com/huggingface/tokenizers), software, version 0.20.3 (released 5 November 2024), the
1302 version used in this study (2024).
- 1303 [31] NLTK Project, Universal Declaration of Human Rights corpus (Unicode
1304 version), NLTK data package udhr2, [https://github.com/nltk/](https://github.com/nltk/nltk_data/blob/550b6625bcef1f2abff2ff770a5a0d272c9c6b2a/packages/corpora/udhr2.zip)
1305 [nltk_data](https://github.com/nltk/nltk_data/blob/550b6625bcef1f2abff2ff770a5a0d272c9c6b2a/packages/corpora/udhr2.zip) commit
1306 550b6625bcef1f2abff2ff770a5a0d272c9c6b2a (1 July 2026), SHA-256
1307 0796c314b09a930c989c6f9d93d226af9af13feccd88496e196c743dd266c7f3;
1308 388 texts from the Unicode Consortium’s “UDHR in Unicode” project
1309 (no longer hosted by the Consortium since January 2024); public domain
1310 (2026).
1311
- 1312 [32] A. Petrov, E. La Malfa, P. Torr, A. Bibi, Language model tokenizers
1313 introduce unfairness between languages, in: *Advances in Neural*
1314 *Information Processing Systems* 36 (NeurIPS 2023), Neural Informa-
1315 tion Processing Systems Foundation, Inc., 2023, pp. 36963–36990.
1316 doi:10.52202/075280–1608.
1317 URL [https://proceedings.neurips.cc/paper_files/paper/2023/](https://proceedings.neurips.cc/paper_files/paper/2023/hash/74bb24dca8334adce292883b4b651eda-Abstract-Conference.html)
1318 [hash/74bb24dca8334adce292883b4b651eda-Abstract-Conference.](https://proceedings.neurips.cc/paper_files/paper/2023/hash/74bb24dca8334adce292883b4b651eda-Abstract-Conference.html)
1319 [html](https://proceedings.neurips.cc/paper_files/paper/2023/hash/74bb24dca8334adce292883b4b651eda-Abstract-Conference.html)
- 1320 [33] K. Whistler, Unicode normalization forms, Unicode Standard Annex
1321 #15, Revision 58 (Unicode 18.0.0), The Unicode Consortium, edited by
1322 K. Whistler. The NFC expansion bound is stated in Section 9, “Detecting
1323 Normalization Forms” (Aug. 2026).
1324 URL <https://www.unicode.org/reports/tr15/tr15-58.html>

- [34] The Coq Development Team, The Coq proof assistant, version 8.18, Zenodo, release V8.18.0 (Sep. 2023). doi:10.5281/zenodo.11551177.
- [35] M. Miculan, KLAIM, certified: Mechanising flow logic for tuple-space coordination, Journal of Logical and Algebraic Methods in Programming 152 (2027) 101179, in press; registered with Crossref September 2026. doi:10.1016/j.jlamp.2026.101179.
- [36] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, A. Rush, Transformers: State-of-the-art natural language processing, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Association for Computational Linguistics, Online, 2020, pp. 38–45, software version used: `transformers` 4.46.3. doi:10.18653/v1/2020.emnlp-demos.6.
- [37] L. Ben Allal, A. Lozhkov, E. Bakouch, G. Martín Blázquez, G. Penedo, L. Tunstall, A. Marafioti, A. Piqueres Lajarín, H. Kydliček, V. Srivastav, J. Lochner, C. Fahlgren, X.-S. Nguyen, B. Burtenshaw, C. Fourier, H. Zhao, H. Larcher, M. Morlon, C. Zakka, C. Raffel, L. von Werra, T. Wolf, SmolLM2: When smol goes big — data-centric training of a fully open small language model, in: Second Conference on Language Modeling (COLM 2025), 2025, preprint: arXiv:2502.02737. URL <https://openreview.net/forum?id=3JiCl2A14H>
- [38] Hugging Face, SmolLM2-135M-Instruct, <https://huggingface.co/HuggingFaceTB/SmolLM2-135M-Instruct>, model weights and tokenizer, revision 12fd25f77366fa6b3b4b768ec3050bf629380bac (last modified 22 September 2025); Apache-2.0 (2025).
- [39] Anthropic, Claude Cookbooks (formerly Anthropic Cookbook), <https://github.com/anthropics/claude-cookbooks>, agent pattern notebooks under `patterns/agents/`, commit 813fbee03cdedfa7808529438d1c7af71f26eb (24 September 2026); repository formerly `anthropics/anthropic-cookbook` (2026).
- [40] LangGraph, <https://github.com/langchain-ai/langgraph>, tutorial notebooks under `docs/docs/tutorials/`, commit 23961cff61a42b52525f3b20b4094d8d2fba1744 (9 January 2026) (2026).

- [41] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, F. Tramèr, AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Datasets and Benchmarks Track, Neural Information Processing Systems Foundation, Inc., 2024, pp. 82895–82920. doi:10.52202/079017-2636. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html
- [42] AgentDojo (source code and recorded runs), <https://github.com/ethz-spylab/agentdojo>, commit 089ed468cf3ed0322acc66b0211f26d9d90dbf60 (2 June 2026) (2026).
- [43] L. Beurer-Kellner, B. Buesser, A.-M. Crețu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Pavard, F. Tramèr, V. Volhejn, Design patterns for securing LLM agents against prompt injections, *arXiv preprint (v3, 27 June 2025)* (2025). arXiv:2506.08837, doi:10.48550/arXiv.2506.08837.
- [44] M. Hofmann, S. Jost, Static prediction of heap space usage for first-order functional programs, in: *Proceedings of the 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL ’03)*, ACM, New York, NY, USA, 2003, pp. 185–197. doi:10.1145/604131.604148.
- [45] J. Hoffmann, K. Aehlig, M. Hofmann, Multivariate amortized resource analysis, *ACM Transactions on Programming Languages and Systems* 34 (3) (2012) 14:1–14:62. doi:10.1145/2362389.2362393.
- [46] W. Qu, M. Gaboardi, D. Garg, Relational cost analysis for functional-imperative programs, *Proceedings of the ACM on Programming Languages* 3 (ICFP) (2019) 1–29. doi:10.1145/3341696.
- [47] J. Bacelar Almeida, M. Barbosa, J. S. Pinto, B. Vieira, Formal verification of side-channel countermeasures using self-composition, *Science of Computer Programming* 78 (7) (2013) 796–812. doi:10.1016/j.scico.2011.10.008.

- [48] J. B. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, M. Emmi, Verifying constant-time implementations, in: 25th USENIX Security Symposium (USENIX Security 16), USENIX Association, Austin, TX, 2016, pp. 53–70. URL <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/almeida>
- [49] T. Æ. Mogensen, Hermes: A reversible language for lightweight encryption, *Science of Computer Programming* 215 (2022) 102746. doi:10.1016/j.scico.2021.102746.
- [50] A. Sabelfeld, D. Sands, Declassification: Dimensions and principles, *Journal of Computer Security* 17 (5) (2009) 517–548. doi:10.3233/JCS-2009-0352.
- [51] C. Manzano, G. de Latorre, A Haskell-embedded DSL for secure information-flow, *Science of Computer Programming* 247 (2026) 103351. doi:10.1016/j.scico.2025.103351.
- [52] N. Ben Said, I. Cristescu, End-to-end information flow security for web services orchestration, *Science of Computer Programming* 187 (2020) 102376. doi:10.1016/j.scico.2019.102376.
- [53] G. C. Necula, Proof-carrying code, in: *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 1997, pp. 106–119. doi:10.1145/263699.263712.
- [54] K. Crary, S. Weirich, Resource bound certification, in: *Proceedings of the 27th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 2000, pp. 184–198. doi:10.1145/325694.325716.
- [55] L. Beurer-Kellner, M. Fischer, M. Vechev, Prompting is programming: A query language for large language models, *Proceedings of the ACM on Programming Languages* 7 (PLDI) (2023) 1946–1969. doi:10.1145/3591300.
- [56] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, C. Potts, DSPy: Compiling declarative language model calls into

- 1424 self-improving pipelines, in: The Twelfth International Conference on
 1425 Learning Representations (ICLR), 2024, arXiv:2310.03714.
 1426 URL <https://openreview.net/forum?id=sY5N0zY50d>
- 1427 [57] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not
 1428 what you’ve signed up for: Compromising real-world LLM-integrated
 1429 applications with indirect prompt injection, in: Proceedings of the 16th
 1430 ACM Workshop on Artificial Intelligence and Security (AISec ’23), ACM,
 1431 New York, NY, USA, 2023, pp. 79–90. doi:10.1145/3605764.3623985.
- 1432 [58] B. Ammanaghatta Shivakumar, S. Priya, P. Gao, AgentFlow: A flow-
 1433 centric policy language and framework for securing LLM agent systems,
 1434 arXiv preprint (v1, 24 August 2026) (2026). arXiv:2608.22868.
- 1435 [59] Y. Cai, W. Tang, C. Wen, S. Qin, Ghost in the agent: Redefining
 1436 information flow tracking for LLM agents, arXiv preprint (v1, 25 April
 1437 2026) (2026). arXiv:2604.23374, doi:10.48550/arXiv.2604.23374.
- 1438 [60] M. Xavier, Vaisakh M A, M. Jolly, M. Xavier, Agentproof: Static
 1439 verification of agent workflow graphs, arXiv preprint (v1, 20 March 2026)
 1440 (2026). arXiv:2603.20356, doi:10.48550/arXiv.2603.20356.
- 1441 [61] S. Khan, Token Budgets: An empirical catalog of 63 LLM-agent budget-
 1442 overrun incidents, with an affine-typed Rust mitigation as a case study,
 1443 arXiv preprint (v1, 2 June 2026) (2026). arXiv:2606.04056, doi:10.
 1444 48550/arXiv.2606.04056.